

Trustworthy agentic diagnosis for industrial processes: evidence-graded competing-hypothesis reasoning evaluated against per-fault literature baselines

ZhangHaoZheng^a

^a*Institute of Advanced Technology, University of Science and Technology of China, Hefei
230088, China*

Abstract

Large language model (LLM) based diagnosis promises root-cause analysis of industrial processes without task-specific training, yet two obstacles keep it out of engineering practice: conclusions that cannot be audited, and evaluation protocols that score sample-level classification while engineers need scenario-level, mechanism-grounded root causes. We present the Industrial Deep Diagnostic (IDD) pipeline: nine-stage agentic orchestration (ontology, deterministic statistics, competing-hypothesis reasoning, physical verification, bounded-repair gates) with evidence grading (L1–L7), an anti-spurious-correlation filter, and a machine-checked evaluation framework of stage-authored, provenance-labelled artifacts. In single recorded executions on a 12-scenario benchmark from three public datasets (SKAB, Tennessee Eastman Process, IndPenSim), IDD resolves 4/9 documented faults to a single root cause (Top-1 44.4%; a hit certifies mechanism-family agreement, never identity, and the three TEP-derived hits remain identifier-recall-possible); the true mechanism is named within the primary finding or surviving-hypothesis set in all nine (top- k 9/9, a set-membership bound); the five unseparable cases return capped `COMPETING_SET` verdicts naming the discriminating measurement; no control raises a false alarm. A model-controlled ablation bounds every accuracy claim: a bare call to the same model family over the same blind digest keyword-scores 9/9 faults (no verdict typing)—more than the pipeline resolves—so the pipeline’s margins are process qualities, not accuracy. One seeded random draw selected a fault scenario for an independent re-execution (one repeat pair), reproducing the verdict state, actuator, and mechanism class within two confidence points; the deterministic suite arms reproduce all 45 output files byte-identically (timestamps excluded); internal consistency is machine-verified.

Keywords: Fault root-cause analysis, Large language models, Multi-agent systems, Competing-hypothesis reasoning, Evidence grading, Reproducible benchmark

Email address: zhanghaozheng@ustc.edu.cn (ZhangHaoZheng)

1. Introduction

Root-cause analysis (RCA) of industrial process anomalies is a knowledge-intensive activity: the engineer must decide *which* of many correlated sensor deviations represents the true underlying cause, justify that judgement with evidence, and communicate it with a calibrated degree of confidence. Three long-standing obstacles keep that automation unsatisfactory in practice.

First, single-LLM diagnosis is unstructured and unauditible. Recent evaluations of LLM-based fault explanation on the Tennessee Eastman Process (TEP) [1] show that large models can articulate plausible mechanisms, but their answers arrive as free-form text with no machine-checkable structure, no separation of facts from inference, and no mechanism that prevents a confident-sounding conclusion from resting on a spurious correlation. A recent review of agentic approaches to process monitoring and fault diagnosis concludes that evidence for LLM/multi-agent fault diagnosis remains scarce [2]: the direction is promising but lacks engineering-grade pipelines.

Second, classical data-driven fault detection and diagnosis (FDD) optimises the wrong target. Principal-component methods [3], deep anomaly detectors [4, 5], and their benchmarks [6] score *per-sample* detection or classification [7]. An engineer, however, asks a *per-episode* question: given this anomalous run, what is the root cause, how confident are we, and what evidence supports it? A 99%-accurate classifier that returns a fault label without mechanism, evidence, or calibration does not answer that question. The detection–diagnosis gap is not merely theoretical: classical detectability studies place the documented disturbances IDV3, IDV9 and IDV15 of the fifteen-fault TEP catalogue at the detection limit, with IDV4 weakly detected by the T^2 statistic [8, 9], and the published LLM baseline’s own PCA implementation leaves all four of these faults unscored [1].

Third, correlation is not causation. In recirculating process loops, almost every pair of variables is correlated through control action or material balance. An RCA method that treats strong correlation as causation will confidently report control-loop artifacts as root causes. Despite decades of causal-inference literature [10, 11], contemporary LLM-diagnosis studies rarely include explicit anti-spurious safeguards.

This paper addresses the three obstacles with the Industrial Deep Diagnostic (IDD) pipeline and a benchmark built to evaluate it honestly. Our contributions are:

1. **A quality-gated agentic diagnosis pipeline** that couples deterministic statistical tooling with LLM reasoning under a competing-hypotheses protocol (Section 4), bounded repair loops, and automatic physical verification, so that every conclusion is decomposable into hypothesis, evidence, exclusion, and confidence.

2. **An evidence-grading and calibration scheme** (levels L1–L7 with gate propagation) that bounds reported confidence by the weakest supporting evidence level. In the corpus none of the DETERMINED verdicts is wrong, and in every scenario whose data cannot separate the surviving mechanisms the pipeline returns a capped COMPETING_SET verdict that keeps the true mechanism ranked and names the discriminating measurement instead of forcing a conclusion (Section 7).
3. **A reproducible evaluation framework**—truth isolation, data fingerprinting, artifact-provenance labels, recorded execution traces, and a machine-checked artifact-consistency gate—in which a scenario is graded only if its complete artifact set was authored by gated pipeline stages and the execution proof passes; an internal integrity audit removed a scripted artifact-fabrication shortcut, and the benchmark was re-executed end to end under the tightened contract (Section 5).
4. **A 12-scenario benchmark** on three public datasets (SKAB, TEP, Ind-PenSim) whose TEP subset carries scored per-fault baselines from Fault-Explainer [1] on four faults (the remaining two lie where its PCA screen excludes them). IDD resolves 4/9 faults to a single root cause while ranking the true mechanism in *every* fault scenario’s surviving set (top- k 9/9) and raising zero false alarms on the controls; a seeded random-draw re-execution reproduced one scenario’s verdict state and mechanism (Section 8.4); and the graded runs executed as true per-stage sub-agent dispatches communicating only through workspace artifacts (Sections 5, 7–8).

Section 2 positions the work against data-driven FDD, LLM-based diagnosis, and multi-agent orchestration research. Sections 3 and 4 detail the architecture and methodology. Section 5 defines the evaluation framework. Sections 6–8 present the benchmark and comparisons, and Section 9 discusses limitations and deployment prospects.

2. Related Work

2.1. Data-Driven Fault Detection and the Benchmark Gap

Statistical process monitoring on the Tennessee Eastman Process established the dominant evaluation template [12, 7]: principal-component analysis (PCA) and projection methods with T^2/Q statistics [3], later extended by deep anomaly detectors [13, 4] and adversarial robustness studies [5]. Within the tripartite partition of fault detection and diagnosis into quantitative-model-based, qualitative-model-based, and process-history methods [14, 15, 16], these families occupy the process-history branch, and the quantitative branch is anchored by model-based analytical redundancy and residual-based fault isolation [17]. All of these branches score *sample-level* detection or *fault-class* accuracy. Critically, they do not evaluate root-cause *mechanisms*: a classifier that labels a run “IDV(4)” provides the class identity but neither a causal mechanism, an evidence trail, nor a confidence statement. The classical detection ceiling (IDV3/9/15 at the limit, IDV4 weak) is documented [8, 9]. Benchmark methodology has drawn

scrutiny: random segment splits inflate reported AUROC—by up to 15.3 percentage points of Macro-AUROC under segmentation-level leakage in Vieira et al. [18], with a lineage formalised by Hendriks et al. [19]—motivating our entity-level scenario splits (Section 5). Scenario-level root-cause localisation itself has a classical lineage that IDD must be read against: Granger-causal and transfer-entropy methods localise propagation directions in plant-wide oscillations [10, 11], signed-directed-graph reasoning performs mechanism-chain elimination [20], and digital-twin practice (surveyed in [21]) mirrors model-based diagnosis.

2.2. LLM-Based Industrial Diagnosis

FaultExplainer [1] is the closest published system: it couples PCA-based detection with LLM-generated fault explanations on TEP. Its evaluation uses two prompt regimes. In the root-causes-included regime the prompt carries the documented root-cause list, the model outputs a top-3 candidate list, and alias matches are accepted (alias classes: IDV1/2/8, IDV3↔IDV9, IDV4↔IDV11↔IDV14, IDV5↔IDV12↔IDV15); here GPT-4o resolves 7/11 and o1-preview 9/11 of the 11 PCA-detectable faults (FaultExplainer’s own PCA implementation excludes IDV3, IDV4, IDV9 and IDV15 from scoring). In its General-Reasoning regime—no candidate list, designed to simulate unseen faults—both models place the correct or a related cause in the top-3 for 8/11 faults under lenient “related” grading (FE’s own text names faults 10 and 13 as failing yet reports 8/11; we transcribe the reported figure). Protocol differences therefore condition the comparison: (i) FE’s outputs are top-3 lists with alias/related acceptance, whereas IDD commits to a single mechanism verdict scored by strict mechanism keywords; (ii) the two PCA-excluded faults in our subset (IDV3, IDV4) are scored by IDD; (iii) FE’s alias policy grades a cooling-water-family answer as correct for IDV11 and IDV14 alike, whereas IDD separates the three forms of the same physical channel. Gong et al. [22] orchestrate multiple LLM agents with difficulty-aware routing, but evaluate on a multiple-choice QA benchmark (FailureSensorIQ [23]) rather than on real time-series data.

Liu et al. [24] combine scene graphs, fine-tuned Llama-3 and augmented reality for machine-tool maintenance, and causal-graph RCA has been studied for time-varying processes [25] and knowledge-map-based group decision settings [26]; a label-free evaluation scheme for diagnosis models has been proposed in the same venue [27]. Multimodal LLM frameworks extend the modality range: Alsaif et al. [28] couple textual, tabular, and image modalities for Industry-4.0 fault detection and diagnosis over industrial fault records, and Merkelbach and Hasenauer [29] show that multimodal LLMs can draft physical models for automated diagnosis, within disclosed limits. Recent work in this journal shows the same momentum and the same gaps. CausalKGPT couples an industrial causal knowledge graph with an LLM for quality-problem cause analysis [30]; FD-LLM fine-tunes a large language model for equipment fault diagnosis [31]; Wen et al. orchestrate LLMs for human-machine collaborative troubleshooting [32]; and Xu et al. pair a deep digital twin with a vision-language model

Table 1: Positioning Against the Closest Published Systems. Each row scores a published or contemporaneous system on real time-series input, scenario-level root-cause analysis, an evidence trail, cap-enforced confidence, and a reproducibility gate; IDD is the only system that combines all five. “Repro. gate” rates whether the system’s evaluation is machine-replayed end to end; PCA/deep FDD baselines are “partial” because detection rates are reproducible but diagnosis claims are not.

System	Input	Scenario-level RCA	Evidence trail	Cap-enforced	Repro. gate
FaultExplainer [1]	TEP series	partial (LLM text)	partial	no	no
AgentRCA [34]	TEP series	yes (ranked list)	partial (traces)	no	no
Gong et al. [22]	MCQA only	no (QA acc.)	no	no	no
PCA / deep FDD (e.g., TranAD [4])	series	no (labels)	no	n/a	partial
Causal RCA (e.g., [26])	series	partial (graph)	partial	no	no
IDD	series	yes (3-state)	yes (L1–L7)	yes	yes (internal)

for multi-scenario fault diagnosis [33]. Closest to our setting, the contemporaneous AgentRCA (July 2026) builds an agentic RCA system that maintains a hypothesis table over tool-grounded evidence, reporting 40.0% Top-1 and 61.5% Top-2 on its TEP evaluation set (it excludes IDV3/9/15 for the classical detectability reasons we discuss [34]; its output is a ranked hypothesis list without calibrated confidence, and its evaluation, like FE’s, stops short of an external truth-comparison harness, and its detectability exclusion also covers the unspecified-cause faults IDV16–20). Table 1 summarises the positioning: among published and contemporaneous systems, none combines real time-series inputs, scenario-level mechanism diagnosis with an evidence trail, cap-enforced three-state conclusions, and a machine-verifiable evaluation contract—the last two, in particular, are absent from every system cited above, with AgentRCA naming uncertainty communication as future work. The claim is scoped to industrial process diagnosis; the contemporaneous agentic-RCA line for software microservices targets a different artifact class (distributed traces and logs) and a different evidence regime.

2.3. Multi-Agent Orchestration Frameworks

Generic agent frameworks (AutoGen [35] and commercial derivatives; ReAct-style reasoning-acting loops [36]) provide message-passing and role prompting but leave audit, evidence policy, and evaluation contracts to the application. Our position, following the practice of the reviewed agentic-FDD literature [2], is that industrial diagnosis requires these as *first-class, machine-checked* pipeline elements rather than prompt conventions; Section 3 describes how IDD enforces them through checkpoints, bounded repair loops, and an event-log execution proof.

2.4. Trust, Automation Levels, and Intellectual Ancestry

IDD’s design decisions sit on well-mapped ground, and we credit the map. The three-state verdict is a level-of-automation and trust-calibration decision—what the system concludes versus what it escalates to humans—in the sense of Parasuraman et al.’s types and levels of automation [37] and Lee

& See’s appropriate-reliance account of trust [38]: a capped `COMPETING_SET` implements calibrated transfer in their sense. The competing-hypotheses discipline descends from Heuer’s Analysis of Competing Hypotheses [39], the four-condition anti-spurious filter from Hill’s causation criteria [40], and the abstention option from Chow’s recognition-with-reject tradeoff [41], whose statistical counterpart is modern selective prediction [42]; the L1–L7 grounding check doubles as a faithfulness proxy in the sense of rigorous XAI evaluation [43]. The named-discriminating-measurement exit is the passive counterpart of *active* fault diagnosis, which designs inputs to separate faults before they occur [44]; IDD inherits the passive setting of plant recording and converts its residual ambiguity into an instrumentation request. The ontology-asset layer continues the process-engineering ontology lineage of OntoCAPE [45]. IDD’s contribution with respect to this ancestry is a domain-specific, machine-checked instantiation of these frameworks inside an agentic pipeline—not their invention.

3. System Architecture

3.1. Execution Architecture

The following terms recur throughout. A *skill package* is a versioned folder holding an agent’s protocol checklist, JSON Schemas, and validation scripts; a *specialist agent* is an independently spawned session that executes one skill package; an *engine* is the CLI/RPC harness that runs such a session. Fig. 1 shows the skill execution architecture as five planes, of which the audit plane is machine-checked. Client surfaces (the web application, CLI drivers, and a CI harness) submit jobs through the engine layer: fourteen interchangeable CLI/RPC engines behind one uniform event contract, with pre-flight probing and typed error codes. The orchestrating main agent holds the domain knowledge as *skill packages* and dispatches each pipeline stage by passing absolute paths (`SKILL_PATH`, `RUN_DIR`); sub-agents communicate only through workspace artifacts, never through conversation context.

Read against the released software, the layer map needs three qualifications. First, the package contains 18 skill packages executed by 14 specialised agents, but the *diagnostic path* reported here (Steps 2–8.5) dispatches eight specialist sub-agents—context-builder, data-processor, diagnostician, judge, report-reviewer, reporter, html-visualizer, html-reviewer—with the VLM visual analyser dispatched by the data-processor inside its Phase 5.5; the remaining packages form optional layers outside the benchmarked path (Section 3.2). Second, a skill package is not a prompt alone: every dispatched package carries the protocol checklist (`references/agent-protocol.md`) and ships the JSON Schemas and validation scripts that machine-check its artifacts before the stage may complete—the structural difference from prompting a model directly. Third, the engine layer gives the pipeline harness portability: all fourteen engines implement one start/stream/close contract, so an engine substitution is a configuration change rather than a re-implementation—pre-registered as an ablation axis, not yet exercised. Every stage transition appends a structured event to

the audit plane, which the log checker and the finalisation gate reduce to a single machine verdict; grading (Section 5) reads only artifacts that survived this plane.

3.2. Knowledge, Web, and Enhancement Layers

Beyond the diagnostic path sit three further layers; how each was exercised, degraded, or left idle in the benchmark is recorded in the run events.

Semantic-knowledge layer. A retrieval-augmented generation microservice [46] (FastAPI + ChromaDB, `rag-retrieval-engine/`) supplies semantic understanding for ontology construction and is the only sanctioned source of level-L6 external evidence; the orchestrator probes it with a three-second health check at Step 0. When it is unavailable, the ontology builder falls back to the shipped `parameter_to_physics.json` mechanism map plus the data’s own self-description, and the degradation is recorded in the run events. In the benchmark corpus the channel was never available: all thirteen runs record a failed probe and build every ontology through the documented fallback (recorded per run as a dedicated probe-failure file or a `rag_validated:false` ontology flag); the TEP ontologies ground their variable semantics in the published process description, cross-checked against the data. No L6-tagged item exists in the released corpus, so no verdict rests on external evidence.

Ontology asset store. Ontologies are published to a portable asset store (`ontology_store.mjs publish`) keyed by a schema fingerprint; a later scenario whose variable schema matches is resolved by deterministic fast-reuse, which copies the validated ontology verbatim into the run directory in seconds instead of re-deriving it. All thirteen benchmark ontologies were published to the store on completion; none was *consumed* by fast-reuse in this corpus, because the benchmark configuration pins the ontology step to full rebuilds (the reuse path is exercised by the production harness outside this benchmark). The store is deterministic and therefore does not affect the benchmark’s truth-isolation contract (it reuses variable semantics, never scenario verdicts).

Integration surface. The web application couples an Express backend with a Vue 3/Vite frontend and an SSE/WebSocket live stream. Its diagnosis endpoint accepts an engine selection and returns typed pre-flight errors for unknown or unusable engines, with probing and the real spawn sharing one code path. The optional enhancement layer (E0–E8, deterministic scripts orchestrated by a dedicated agent) post-processes completed runs for reporting depth and lies outside the benchmarked path.

3.3. Pipeline Overview

IDD organises diagnosis as a nine-stage pipeline executed by the specialised agents backed by the skill packages (Fig. 2; the benchmark’s execution form—true per-stage sub-agent dispatch—is disclosed in Section 5). The main orchestrator performs setup and data ingestion (Step 0/1) with adaptive pre-processing; a context-builder agent constructs a process ontology with variable semantics, physical principles, and confounder annotations (Step 2, checkpoint

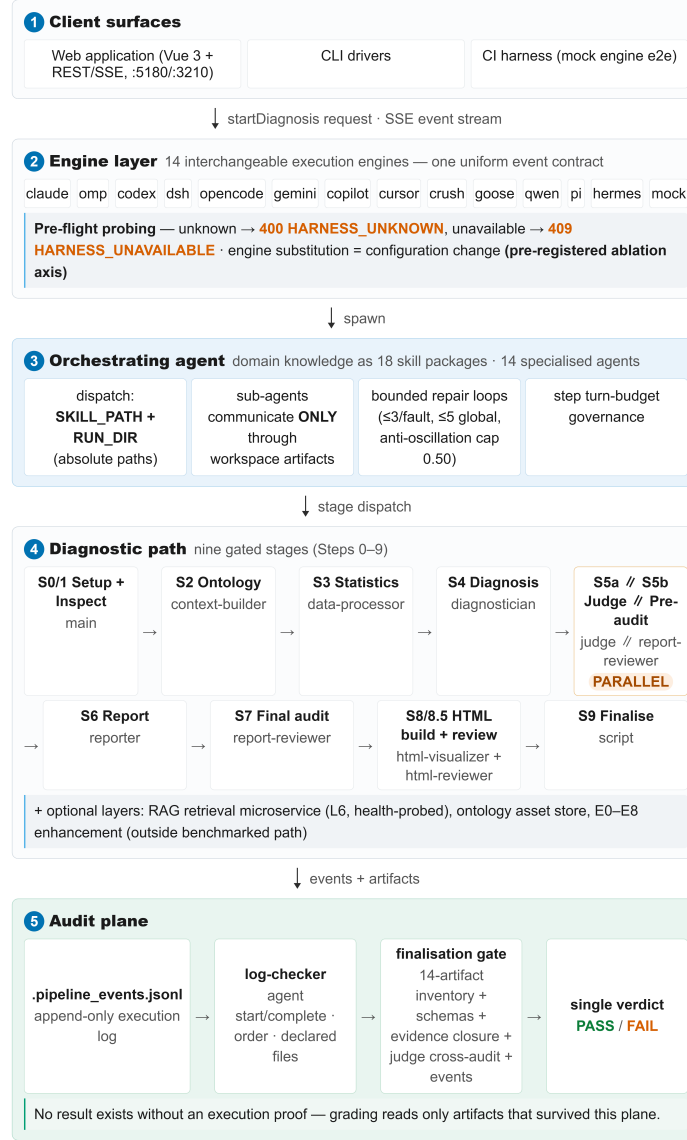


Figure 1: Skill execution architecture (HTML-rendered diagram; source and export script released with the paper). Fourteen interchangeable engines expose one uniform event contract to the orchestrating agent, which dispatches the diagnostic path’s eight specialist sub-agents from 18 skill packages; sub-agents communicate only through workspace artifacts, and the audit plane’s log checker and finalisation gate produce the execution proof the benchmark grader consumes.

CP-2/3); a data-processor agent runs the deterministic statistical stage (per-variable anomaly profiling, cross-domain correlation, lag-compensated cross-

Table 2: Module contracts of the diagnostic path: every stage, the agent that authors its artifacts, the artifacts it consumes and produces (filenames as released; all paths are relative to the run directory), and its checkpoint. The table lists each stage’s contracted deliverables; runs additionally hold auxiliary stage-authored files. Checkpoint predicates: Table 3.

Stage	Agent	Consumes	Produces	Gate
Setup/inspect (0/1)	main agent	scenario CSV, user context	00_input/input_manifest.json, user_context.json, run_config.json	CP-1
Ontology (2)	context-builder	input_manifest.json, RAG digest, process description	01_ontology/ontology.json (+00_input/clarification_needed.json)	CP-2/3
Statistics (3)	data-processor	ontology.json, cleaned data	02_processed/data_analysis_conclusion.json (mandatory handoff), validate_report.json, 03_figures/03_figures/visual_analysis.json	CP-4
Visual evidence (3.5)	vlm-visual-analyzer (inside data-processor Phase 5.5)	03_figures/, image_captions.json		—
Diagnosis (4)	diagnostician	analysis conclusion, ontology, digests	04_diagnostics/{diagnosis, evidence, confidence, reasoning_chain}.json	CP-5
Judge pre-audit (5a/5b)	judge report-reviewer	04_diagnostics/*	05_review/judge_feedback.json 05_review/optimizer_preflight.md	CP-6
Report (6)	reporter	all upstream artifacts	report.md, run_summary.json	CP-7
Final audit (7)	report-reviewer	report.md, artifacts	optimizer.md containing ENDORSED	CP-8
HTML build (8)	html-visualizer	artifacts, design system	render_manifest.json → diagnostic-report.html → html_selfcheck.json	—
HTML review (8.5)	html-reviewer	page + manifest	05_review/html_review.json (verdict=pass)	CP-9
Finalise (9)	deterministic script	all of the above	pipeline_finalize_report.json (PASS), evidence_closure_report.json	—

correlation, batch integrity checks), producing a machine-readable analysis conclusion (Step 3, CP-4); a vision–language-model (VLM) agent reads rendered charts as visual evidence (pipeline Step 3.5 executes inside the data-processor agent as its internal Phase 5.5; in this deployment it executed its documented non-visual fallback in every run, Section 6). The diagnostician agent then performs competing-hypothesis reasoning over the statistical and ontological evidence (Step 4, CP-5). Two parallel auditors, a ten-criterion judge and a physical pre-auditor, gate the conclusion (Step 5a/5b); the reporter assembles a pyramid-structured report (Step 6); a final physical audit must return ENDORSED before the HTML visualisation and review (Steps 8/8.5, CP-9 at the review gate) and the evidence-closure finalisation (Step 9).

Why nine stages. Three rules govern the decomposition; the model-controlled ablation (Section 8.3) independently supports each. Deterministic work is pushed down to scripts: statistics, hashing, schema checks, and the finalisation inventory run as versioned code so they are byte-reproducible, and only semantic judgement (ontology, hypotheses, causal framing, prose) is left to agents—only this split keeps the deterministic layer defensible once a bare model call is known to be accurate. Data description precedes diagnosis: the ontology and the statistics digest are produced before any hypothesis is generated, which is what forces the diagnose-only-what-the-data-supports behaviour observed in the capped verdicts. Audit and delivery are gated last and separately: two independent auditors (judge and physical auditor, the pipeline’s

Table 3: Checkpoint gates CP-1...CP-9. Every predicate is mechanical—file existence, JSON-schema validity, or a numeric lower bound—so the gates can be re-run by a reader without domain judgement; a failing predicate blocks the transition and routes to the stated repair action.

CP	Transition	Machine predicate	Failure action
CP-1	1→2	<code>input_manifest.json</code> + <code>user_context.json</code> + <code>run_config.json</code> exist	back to Step 0
CP-2	2→2.5	<code>ontology.json</code> ≥1 KB and schema-valid	re-run ontology builder
CP-3	2.5→3	<code>clarification_status</code> ∈ { <code>AUTO_RESOLVED</code> , <code>USER_CONFIRMED</code> }	resolve clarification
CP-4	3→4	<code>data_analysis_conclusion.json</code> present and generated plots > 0	re-run data-processor
CP-5	4→5	four diagnosis outputs schema-valid and quality-checked	repair diagnosis
CP-6	5→6	judge repair summary written; pre-audit reports no FATAL finding	repair (best-of-3)
CP-7	6→7	<code>report.md</code> + <code>run_summary.json</code> present	re-run reporter
CP-8	7→8	<code>optimizer.md</code> contains <code>ENDORSED</code>	repair loop
CP-9	8.5	<code>diagnostic-report.html</code> ≥5120 B and review verdict <code>pass</code>	re-run html-visualizer

only mandatory parallel step) review the reasoning before a report exists, and the HTML deliverable is built manifest-first and reviewed independently, so the final artifacts cannot diverge from the audited verdict. The stage boundaries themselves are the artifact handoffs in Table 2, and the mechanical predicates of the checkpoints are listed in Table 3.

3.4. Execution Layer: Interchangeable Engines With a Uniform Contract

Reasoning agents are executed by any of fourteen interchangeable engines—the Claude Agent SDK, several RPC and ACP dialects, and a set of one-shot CLIs—behind a single start/stream/close contract realised as a capability table (`engines.mjs`) plus first-party wiring entries in the service dispatch table. Engine availability is probed and validated before execution with typed error codes (400 unknown engine, 409 `HARNESS_UNAVAILABLE` unusable engine); a scripted in-process mock engine exercises the full pipeline in CI without external dependencies. Because every engine emits the same standardised event stream, the benchmark harness consumes any compliant engine’s stream identically; diagnostic quality across engines is an empirical ablation question, and engine substitution remains a designed but not-yet-exercised ablation axis (Section 9).

3.5. Audit Infrastructure

Every stage transition appends a structured event to `.pipeline_events.jsonl`; a log checker validates the start and completion records of every agent against the required contract, and a finalisation gate aggregates inventory, schema, closure, judge-cross-audit, and event-archive checks into a single PASS/FAIL verdict. A run that fails any critical check cannot be graded, which is the property the benchmark relies on: *no result exists without an execution proof*. The gate is exercised, not decorative. The released records show the round-1 judge failing TEP IDV3 at 75 with two blocking issues, repaired to a round-2 pass at 90 under the declared scope; the final physical audit caught two report-layer defects (a mis-attributed row reference and a residual over-claim in a run summary), repaired and re-audited; the grading rubric (Section 6) records

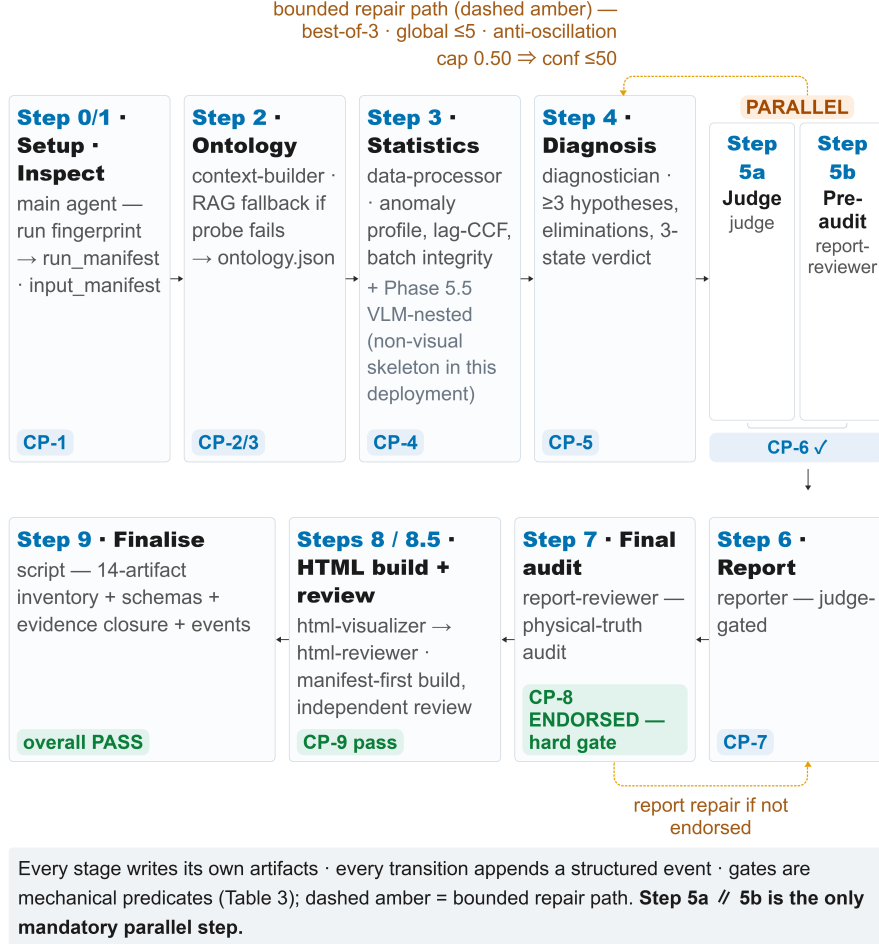


Figure 2: Diagnostic pipeline flow for one scenario (HTML-rendered diagram; source and export script released with the paper). Solid arrows: strictly sequential stages, each gated by a machine-validated checkpoint (blue tags); the judge and the physical pre-audit (Steps 5a || 5b) are the only mandatory parallel stages, drawn side by side feeding one gate. The dashed amber edges are the bounded repair loops: a sub-threshold judge score re-enters the gated diagnosis stages under the bounded-repair counters of Section 4, and a final audit without endorsement returns the report to Step 6—never an unbounded cycle. The second row reads right-to-left (serpentine flow); the return arrow at the row boundary marks the direction. Green boxes mark the hard gates: the final audit must return ENDORSED (CP-8) and the finalisation gate must return PASS (Step 9) before the deliverables exist.

calibration and grounding deductions in three further runs; and several runs additionally bounced stage artifacts for schema or path-normalisation issues before passing. Every rejection was repaired by the artifact’s owning stage and re-gated to PASS; no verdict text changed in any of these repairs. Events are

pipeline-internal records, not externally signed attestations; the resulting trust boundary and its limits are stated in Section 9.

4. Diagnostic Methodology

4.1. Problem Formulation

A scenario supplies a recorded window D_S of p process variables and, optionally, user context; the task is to produce a verdict $v \in \{\text{DET}, \text{CS}, \text{ND}\}$ with a mechanism-level root cause, a confidence $c \in [0, 1]$, and an evidence chain. The diagnostician forms a hypothesis set $H = \{h_1, \dots, h_k\}$, $k \geq 3$, where each h_i carries a mechanism class m_i , a set E_i of evidence items, and a contradiction set X_i . Two definitions govern the protocol. The *evidence level* of a hypothesis is the weakest level in its chain, $\ell(h_i) = \min_{e \in E_i} \text{level}(e)$ with $\text{level}(e) \in \{L1, \dots, L7\}$ (Section 4.3)—a conclusion cannot be stronger than its weakest citation. The *confidence ceiling* is the minimum of the five-factor assessment and every protocol cap applicable to the verdict state,

$$c \leq \bar{c}(v) = \min(F, \mathcal{C}(v)), \quad (1)$$

where F is the five-factor assessment and the cap function $\mathcal{C}(v)$ evaluates to the smallest applicable protocol cap: 0.70 for CS, tightened to 0.65 for its indistinguishable subclass, and 0.50 whenever the anti-oscillation rule (code name `PARAM_AMBIGUITY`) is active—all three are documented in the agent-side protocol. The agent-side protocol documents no DET cap, but the machine grading gate flags—rather than blocks—confidence above 0.90 for every non-CS state and above 0.70 for CS (calibration check R4 of the released rubric; the two 0.91/0.93 verdicts of Section 7.2 are exactly this flag firing), so the effective cap set $\mathcal{C}$ includes 0.90 for DET at the gate (the gate’s flat non-CS 0.90 ceiling also covers ND). Reason-coded per-run ceilings may still bind; the released schema enumerates the reasons: indistinguishable competing set, unsorted-data lag claims, product-specific-only evidence, all-criteria-verified, direct measurement. For example, the canonical IDV7 run binds its DETERMINED verdict at 0.78 under a reason-coded ceiling of 0.80 (recorded reason `ALL_CRITERIA_VERIFIED`, noting that the root variable, header supply pressure, is unobservable, so the verdict is capped at the justified-inference level). Elimination is quantitative: h_i is eliminated when its contradictions outweigh its support, $|X_i| > |E_i|$ with each item weighted by its evidence level (weights in the released protocol; the machine-checked floor is ≥ 2 eliminations, exclusion confidence ≥ 0.90 , and a citation per elimination), and every elimination records a revival condition. The three-state verdict is then the decision function

$$v = \begin{cases} \text{DET}, & |H_{\text{surv}}| = 1 \text{ and every elimination is evidenced,} \\ \text{CS}, & |H_{\text{surv}}| \geq 2 \text{ and no channel separates the survivors,} \\ \text{ND}, & \ell(h^*) < L5 \text{ or a required measurement is absent,} \end{cases} \quad (2)$$

with the reported confidence satisfying $c \leq \bar{c}(v)$ (Eq. (1)). The observed 0.55–0.63 band of capped verdicts reflects this INDISTINGUISHABLE subclass. The

Algorithm 1 IDD diagnosis of scenario S (script = deterministic code, agent = semantic reasoning; gates as in Table 3)

- 1: **script:** fingerprint D_S , compute statistics digest, write blind brief $\triangleright$ Step 0/1
 - 2: **agent:** $O \leftarrow$ ontology (brief) or **script**-side fast-reuse from the asset store $\triangleright$ Step 2, CP-2/3
 - 3: **agent:** $A \leftarrow$ data-processor (D_S, O); plots, validation report $\triangleright$ Step 3, CP-4
 - 4: **agent:** form H , attach (E_i, X_i) with evidence levels $\triangleright$ Step 4
 - 5: **agent:** eliminate while $|X_i| > |E_i|$; compute v by Eq. (2)
 - 6: $c \leftarrow \min(\text{five-factor assessment}, \bar{c}(v)) \triangleright$ Eq. (1)
 - 7: **if** judge score < 90 **then snapshot** best round; repair in declared scope (≤ 3 rounds, global cap 5)
 - 8: **end if** $\triangleright$ CP-6
 - 9: **agent:** report; **verify:** physical audit must return ENDORSED $\triangleright$ Step 6/7, CP-7/8
 - 10: **agent:** HTML manifest-first, then independent review $\triangleright$ Step 8/8.5, CP-9
 - 11: **script:** finalise—the grader’s 14-artifact completeness set, schemas, closure, event log $\Rightarrow$ PASS; else the run cannot be graded (the Step-9 finaliser separately executes its own ≈ 45 -check inventory)
-

five-factor assessment is the protocol’s confidence decomposition over statistical strength, physical plausibility, temporal evidence, absence of confounds, and symptom completeness (weighted 25/25/20/20/10, totalling 100, in the released `confidence.json`); the three states are protocol terms throughout this paper and carry no everyday meaning: DET asserts a single evidenced mechanism, CS asserts that the present data cannot separate the survivors and names the measurement that would, and ND declares the evidence insufficient. Algorithm 1 states the whole procedure as executed, with deterministic (script) and semantic (agent) steps marked; the ablation of Section 8.3 is the empirical test of this split.

4.2. Competing-Hypotheses Protocol

The diagnostician agent must generate at least three candidate hypotheses per fault scenario (the protocol’s intellectual ancestry is credited in Section 2.4), assign each a mechanism class (e.g., fluid restriction, cavitation, rotor dynamics, feed-composition step, cooling disturbance, sensor drift), collect for each an evidence list with explicit levels, identify contradictions, and eliminate hypotheses whose contradictions are stronger than their support. The protocol admits exactly three conclusion states: DETERMINED (a single surviving hypothesis, bounded by the five-factor assessment; reason-coded per-run ceilings may further bind it), COMPETING_SET (≥ 2 survivors that the data cannot discriminate, with confidence capped at 0.70; observed caps in this corpus are 0.55–0.63 and the discriminating measurement is named), and NEEDS_DATA (evidence insufficient; missing data itemised). A fourth, implicit state—normal operation with

no confirmed fault—applies to control scenarios. The elimination contract is quantitative: every elimination must cite the contradicting evidence, and every surviving chain must cite evidence numbers present in the statistical digests.

4.3. Evidence Grading and Gate Propagation

Every piece of evidence carries a level L1–L7 (L1 direct measurement, L2 operating documents, L3 statistical analysis, L4 visual evidence, L5 domain ontology/mechanism, L6 external reference, L7 unsupported assumption). The ordering is a policy choice: direct measurement outranks statistical processing because a sensor reading cannot inherit a processing error, while visual evidence is graded below statistics because chart reading is itself an inference. The scenario confidence is bounded by the weakest level contributing to the surviving chain, and the quality gate flags, for deduction, conclusions whose confidence exceeds that bound. This converts “hallucination control” from a prompt instruction into a checkable arithmetic constraint.

4.4. Anti-Spurious-Correlation Filter

Four conditions must hold before a correlation may be cited as causal support: (i) temporal precedence of cause over effect, verified by lag-compensated cross-correlation; (ii) statistical significance surviving detrending, Simpson-stratification checks, and multiple-testing correction; (iii) a physical mechanism annotated in the ontology connecting the variables; and (iv) absence of contradictions under leave-one-out outlier sensitivity analysis, with production-state (steady/batch-phase) filtering applied before correlation. Correlations failing the filter are recorded but explicitly barred from hypothesis support; definitional pairs (e.g., valve position and the level it controls, $r = 1.0$) are classified as control-loop artifacts. In the benchmark this filter is the reason control-loop correlations are not reported as root causes, and the reason normal batches with impulsive recipe-dosing spikes ($|z|$ up to 16.2) are graded as healthy rather than faulty.

4.5. Automatic Physical Verification

After the conclusion is drafted, a physics checker recomputes quantitative feasibility (mass/energy balances, thermal and vibration magnitude envelopes) from the cleaned data, ontology, and anomaly report, producing a machine-readable `physical_checks` verdict. When no automated check applies to a scenario’s column semantics, the pipeline falls back to an explicit L1–L5 manual-verification memo; silent omission is not permitted.

4.6. Quality Gates, Repair Loops, and Calibration

A ten-criterion judge scores the diagnosis (threshold 90); a parallel physical pre-audit reviews mechanism chains; failures trigger bounded repair (per-fault rerun ≤ 3 , global cap 5, with anti-oscillation confidence capping at 0.50). The final physical audit must return ENDORSED. If the review-repair budget exhausts without an endorsement, the run proceeds best-effort and the finalisation

gate records the missing endorsement as a critical failure, so the run is a pipeline failure rather than a gradeable result. Ceiling compliance is then auditable end-to-end: every scenario grading records whether the reported confidence respects the protocol ceiling, and *overconfidence* (DETERMINED *and* wrong) is a first-class error metric.

The released governance keeps a stochastic reasoner bounded by design. Each repair round snapshots the best-scoring attempt (`best_round` copy of the diagnosis set); the second and third rounds run in a declared *repair scope* that recomputes only the implicated artifacts and marks the rest as carried over, so a fix cannot silently rewrite evidence that was not found defective (auditable from the declared repair scopes and carried-over markers in the released logs). The loop never halts the run on score alone: if the threshold is still unmet after the budget, the pipeline delivers a best-effort report with a gate-enforced capped verdict (and the cap recorded), because failing to deliver is worse than delivering with a disclosed confidence. The protocol adds two machine-checkable disciplines. The *data-truth mandate* requires every number in a diagnosis to be reproducible from the raw data (chain items carry derived markers and a NOT-APPLICABLE fallback), and the red-light list makes the principal failure modes auditable: no forced single verdict from a competing set, no global correlation cited as causal evidence, no conclusion without an evidence-level tag, no HTML outside the manifest. Violations are what the judge is instructed to flag, which is how the audit plane of Section 3 remains a check rather than a claim.

5. A Reproducible Evaluation Framework

IDD’s benchmark framework (released as the project’s **experience/** evidence repository and **docs/benchmark/** protocol) is designed around five contracts, each machine-enforced:

(C1) Truth isolation. Scenario definitions carry ground truth and grading keywords, but the pipeline’s visible inputs contain only the process description and the data. Ground truth never enters the inference context; the grader owns it exclusively. A leakage sentinel machine-checks the contract from both directions: no ground-truth string may appear in any agent-visible brief, and no fault-vocabulary may appear in the run directories’ user-context files. Scenario definitions—including the grading keywords and the allowed verdict set of every scenario—are versioned in the released repository with their edit history. The keyword sets used to grade the current corpus are the calibrated set pinned at commit `221d061`—over-generic tokens removed, mechanism synonyms added—then frozen before any current-era execution. The calibration is truth-adjacent in two ways that Section 7.2 quantifies: the “v1-era corpus” it was tuned against denotes the pre-discipline execution batch of the same twelve scenarios (run directories dated before the 2026-09-14 anti-oscillation and confidence-cap revision, retained with the released edit history), and the published per-fault baselines contributed the mechanism synonyms. The sentinel machine-checks briefs and user-context files, not run-directory names; the identifier channel is the residual disclosure channel and is treated in Section 9. These keys are

author-designed (Section 9): a hit certifies mechanism-family agreement with the documented truth, not identity, and one scenario’s set includes a generic token.

(C2) Data fingerprinting. Every input file is hashed (SHA-256) into a dataset manifest; the reproducibility gate re-verifies all 153 input fingerprints and the leakage sentinel returns a PASS verdict before any metric is accepted.

(C3) Stage-authored artifacts with provenance. Each pipeline stage writes its own machine-checkable artifacts: the ontology, the statistical analysis conclusion, the four-piece diagnosis, the judge’s ten-criterion feedback, the auditors’ ENDORSED verdicts, the report, the page model (`render_manifest.json`) behind the HTML deliverable, and the reviewer’s independent verdict. Every scored verdict carries an explicit provenance label (`judge-agent-10-criteria`, `auditor-agent`, `html-reviewer-agent`), so downstream consumers can distinguish independently authored judgements from rendered ones.

(C4) Gate-graded scoring. The grader accepts a scenario only if its run directory contains the complete artifact set of an executed pipeline (the fourteen items in Table 4), the event log passes the log checker, and the finalisation gate returns PASS. Otherwise the scenario is reported as *not executed*—a pipeline failure, not a benchmark result. The gate is exercised in practice: an earlier protocol revision allowed a scripted shortcut that expanded a single reasoning note into a full artifact set (complete with synthesised agent events); the internal integrity audit flagged the fabrication, the shortcut was deleted from the codebase, and the entire benchmark was re-executed through the real pipeline. The grader’s completeness gate makes the shortcut permanently unreplayable.

(C5) Artifact-consistency gate. A verifier recomputes all aggregate metrics from the per-scenario gradings, re-checks dataset hashes, verifies case coverage, and requires an execution proof per run; any drift fails the evaluation. The gate verifies the internal consistency of the recorded execution—recomputed aggregates, hashes, coverage, proofs—and is deliberately not an independent re-execution of the reasoning layer; the released report carries the resulting internally-consistent verdict.

Execution form. The canonical benchmark runs reported here were executed in the production dispatch form: an orchestrating agent dispatched each pipeline stage to an *independent sub-agent session* through the harness’s sub-agent spawn tool, passing absolute paths (`SKILL_PATH`, `RUN_DIR`); sub-agents communicated only through workspace artifacts, never through conversation context, and each authored its own artifacts under its own skill protocol. The pipeline’s one mandatory parallel step (judge and pre-auditor, Steps 5a/5b) executed as two concurrent sessions. The orchestrator’s role was limited to dispatch, gate verification, and event bookkeeping—it authored no pipeline artifact and no agent event. The benchmark driver itself retains the anti-fabrication division of labour: its scripts verify and score; where artifacts were missing it printed an execution contract and refused to grade. The event logs record every agent start/completion with declared outputs, and the log checker validates ordering and declared files per run. This form is the property that makes the

Table 4: The graded artifact set enforced by the completeness gate (C4). A run missing any item is *not executed* by the grader—a pipeline failure, not a benchmark result.

#	Run path	Role
1	01_ontology/ontology.json	process ontology (variables, semantics, mechanisms)
2	02_processed/data_analysis_conclusion.json	statistical handoff to the diagnostician
3	04_diagnostics/diagnosis.json	verdict, hypotheses, eliminations
4	04_diagnostics/evidence.json	evidence inventory with numeric evidence ranks (1–7, the L1–L7 levels)
5	04_diagnostics/confidence.json	per-hypothesis confidence and ceiling audit
6	04_diagnostics/reasoning_chain.json	R1–R8 reasoning chain with falsification conditions
7	05_review/judge_feedback.json	judge sub-agent ten-criterion verdict
8	05_review/optimizer_preflight.md	pre-report physical audit
9	05_review/html_review.json	independent HTML review (verdict=pass)
10	report.md	pyramid report
11	run_summary.json	machine-readable run summary (consistency-checked)
12	render_manifest.json	page model, written before the HTML
13	html_selfcheck.json	HTML self-check against the manifest
14	diagnostic-report.html	HTML deliverable (size re-check rejects files below 5 120 B)

audit plane meaningful: had stages shared one session, the artifact contracts and gates would be unchanged, but the independence between the authoring stage and its reviewers would be weaker than the production design intends. The independence is session- and protocol-level, not model-level: every stage runs on the one model family disclosed in Section 6, so correlated failures between a generator and its reviewer remain possible.

Table 4 lists the graded artifact set the completeness gate enforces; the same fourteen filenames appear in the released run directories, so “the complete artifact set” is a checkable list rather than a phrase.

Reproducibility semantics. The framework distinguishes two layers explicitly. The deterministic statistical stage is byte-reproducible: identical inputs and code regenerate identical digests, and the released dataset fingerprints pin the inputs. The reasoning layer is reproducible at the *procedure* level: re-executing the protocol re-launches the same gated pipeline on the same blind inputs and re-derives conclusions under the same gates and truth comparison; wording may vary between runs, but every verdict remains bound to the same evidence discipline. Reproduction claims in this paper therefore distinguish the two layers, and bit-level reproducibility is claimed only for the deterministic stage.

Scenario splits follow the entity-level discipline recommended by recent benchmark studies [18, 19]: each experimental batch/file appears in exactly one scenario, controls are paired with faults per dataset, and no segment-level splitting is used anywhere.

6. Experimental Setup

6.1. Datasets and Scenarios

We assemble 12 scenarios from three public benchmarks, listed in Table 5. **SKAB** [47] is a closed water-circulation testbed with vibration, current, pressure, temperature, and flow channels at 1 Hz. Its two fault scenarios are inlet-valve throttling (the official label file names suction-line valve restriction) and

two-phase-flow cavitation drawn from the “other” group, whose official label names cavitation but is withheld from the scenario description; the anomaly-free control completes the set. **TEP** [12, 48, 6] contributes six documented fault scenarios: IDV1 feed-composition step, IDV3 D-feed-temperature step (classically hard to detect), IDV4 reactor-cooling-water step, IDV7 C-header pressure loss, IDV11 random cooling-water variation, and IDV14 cooling-valve stiction, plus the fault-free baseline as control. The TEP scenario briefs declare the anomaly-onset window (sample 161, idx 160 zero-based), so detection is out of scope for these scenarios and the task is RCA within a localised window; the IndPenSim briefs declare no onset. **IndPenSim** [49, 50] is a 100,000 L penicillin fermentation simulator; its documented process-deviation batch is 93, whose logged deviation is actuator-side (out-of-band pH and temperature excursions driven by the acid-dosing and heating-water flows—a description taken from the benchmark case definition, not from the paper’s diagnosis), with one normal batch as control; the pipeline’s cooling-loop signature for batch 93 is evaluated against this documented deviation in Section 7. Ground truth derives from verifiable sources only: SKAB’s official label file, the established TEP cause table [9], and IndPenSim’s faulted-batch markers. Scenario selection followed a documentation rule rather than random sampling: every scenario corresponds to a publicly documented fault episode, chosen so that each dataset contributes mechanisms from distinct families plus a paired control; the consequences of non-random selection are discussed in Section 9. The control briefs mirror the fault briefs’ structure but differ in what they disclose: the SKAB control’s process description is identical to its fault sibling’s except that the fault brief announces an anomaly (onset withheld), while the TEP and IndPenSim control briefs state that the segment is the fault-free baseline—just as the corresponding fault briefs state that an anomaly exists. Only the SKAB control is therefore blind: the zero-false-alarm outcome shows that the pipeline does not fabricate faults under an honest brief, and the SKAB control additionally exercises blind discrimination (its strong raw correlations are refuted, Section 7.4).

Three design decisions shaped the subset. First, the TEP subset is chosen so that *every fault maps onto a published per-fault outcome* from FaultExplainer [1], enabling scenario-granular comparison rather than aggregate-only comparison: four faults (IDV1/7/11/14) carry scored baselines, and IDD additionally scores the two faults that screen excludes, IDV3 and IDV4. Second, IDV3 sits at the classical PCA detection limit and IDV4 is weakly detected [8, 9]—both lie where classical detection and the published baseline stop—so they probe root-cause reasoning at and beyond that limit. Third, IDV4, IDV11 and IDV14 perturb the same physical channel (reactor cooling water) with three different dynamic forms (step, random variation, valve stiction); separating them requires reading the statistics of each recording, not recalling a fault label, which partially mitigates prompt-knowledge contamination.

6.2. Metrics

For fault scenarios we report **resolved Top-1** (a DETERMINED verdict naming the true mechanism; coincides with the FDD community’s correct-diagnosis-

Table 5: Benchmark Composition. Twelve scenarios from three public datasets: each dataset contributes its documented fault scenarios plus one anomaly-free control, with ground truth drawn from official label files, the established TEP cause table, and IndPenSim’s faulted-batch markers.

Dataset	Fault	Control	Truth source
SKAB	2	1	official <code>ground_truth.json</code> (label withheld for the “other”-group case)
TEP	6	1	Downs–Vogel; Chiang et al. [12, 9]
IndPenSim	1	1	faulted-batch markers (batch 93)
Total	9	3	—

rate, CDR), **top- k** (the true mechanism is named within the primary finding or the surviving-hypothesis set that the grader reads; because the protocol mandates at least three generated hypotheses, this approximates candidate-list membership, and for a single-survivor DETERMINED verdict it coincides with resolved Top-1; reported alongside, not instead of, resolved Top-1), **ceiling compliance** (reported confidence within the protocol ceiling of Section 4), **verdict-type compliance** (conclusion type within the scenario’s allowed set; a DETERMINED-and-wrong verdict counts as *overconfidence*), and a **protocol-compliance rubric score**—a deterministic, machine-computed checklist (100 points) computed from the pipeline artifacts themselves and measuring conformance to this paper’s own protocol rather than correctness (see Section 9). Top- k is structurally favoured when several scenarios in a family share keyword tokens (the three reactor-cooling TEP faults share “reactor cooling”); read it as family-level membership, not specific-form identification. The strictest, branch-localised reading would count IDV3 as a miss (its D/E-feed-temperature survivor sits on the stripper-feed branch while the documented truth is the reactor-feed D stream), giving top- k 8/9 under that reading—both readings are reported. For control scenarios we report **pass** (no confirmed fault) and **false alarms**. Proportions carry Wilson 95% confidence intervals at $z=1.96$ (the released scorer’s constant). We do not report sample-level F1/AUC: they answer a different question and are used only as reference context (comparison protocol below).

The rubric’s seven checks are computed strictly from the stage-authored run artifacts: **R1** artifact integrity (ontology, analysis conclusion, and the schema-valid diagnosis four-piece; 15 points), **R2** hypothesis structure (≥ 3 hypotheses, ≥ 2 eliminations each citing substantive contradictory evidence; 15), **R3** evidence grounding (every surviving chain cites at least one number present in the scenario’s own evidence brief; 20; fault scenarios—the check has no fault-relevant targets in a control brief, and the one control deduction under it is recorded as a check mis-specification, not a run defect), **R4** calibration (confidence within the protocol ceiling; conclusion type within the scenario’s allowed set; 15), **R5** falsifiability (a surviving hypothesis names the observation that would revise it; 10), **R6** physics verification (automated check valid or explicit manual memo; 10), **R7** execution proof (event log + finalisation PASS + HTML review gate; 15). The pipeline’s internal ten-criterion judge remains the in-pipeline gate (Section 4)—its scores are recorded with provenance and reported as execution-

quality evidence—but it is *not* a benchmark metric: benchmark quality scoring is fully mechanical. The rubric’s scope is limited in two ways. First, it is a conformance instrument with limited discriminative power: nine of the twelve scenarios score at ceiling because the structural checks cannot fail for any run that has passed the gates—correctness claims rest on the truth comparison, never on the rubric. Second, “strict mechanism-keyword hit” is not mechanism equivalence: the keyword sets are disclosed per scenario, one of them (the Ind-PenSim process-deviation scenario) includes the generic token “deviation”, and a hit certifies mechanism-family agreement, not identity. The consequences are quantified in Section 7, including an identity-strict Top-1 count that discounts the batch-93 verdict.¹

6.3. Comparison Protocol

We distinguish two comparison registers. **Register A (same-task direct comparison)**: methods that output one root-cause conclusion per scenario on the same dataset family; the published FaultExplainer per-fault numbers fall here, at scenario granularity. **Register B (reference only)**: sample-level detection/classification results; they are reported for context and must not be ranked against our scenario-level metrics. Register A comparisons inherit two protocol differences: FaultExplainer supplies candidate root causes in its prompt and accepts alias matches, whereas IDD uses no candidates and scores mechanism keywords strictly; and FaultExplainer scores only the 11 PCA-detectable faults, whereas IDD additionally scores the two faults that screen excludes, IDV3/IDV4. Because the scenario sets differ, no significance test is claimed across systems.

6.4. Implementation

The deterministic stages run as versioned scripts (streaming CSV inspection, anomaly profiling, correlation with lag-compensated cross-correlation functions (CCF), validation with Simpson/detrending/outlier-sensitivity/multiple-testing checks, physics checking via the shared scientific Python environment). The reasoning stages execute the full nine-stage agent pipeline under their skill protocols (Section 3); every stage transition is recorded in the event log. The twelve canonical scenarios were executed as concurrent batches under one orchestrating agent (per-run event spans reflect orchestration batching, not isolated agent time); the clean per-scenario figure is the fully sequential random-draw re-execution (Section 8.4; ≈ 2.0 h end to end). The deterministic stage takes seconds per scenario. The entire chain (prepare, briefs, pipeline-execution verification, gate-graded scoring, rubric, aggregation, artifact-consistency gate, report) is driven by a single entry point with fail-fast stage gates.

¹Artifact-field semantics for auditors: the released `metrics.json` folds verdict-type compliance into its `calibrated` flag, and `rubric.json` records the SKAB-valve type miss under its calibration-check key (the failing payload is the allowed-set violation described in Section 7.4); Table 6 reports calibration and verdict-type compliance as separate metrics.

Benchmark configuration disclosure. All thirteen runs reported here (twelve canonical scenarios plus one random-draw re-execution) were executed on one harness: the ZCode agent CLI (the CLI-driver class of the engine layer, dispatching each stage to an independent sub-agent session; execution form in Section 5) with the GLM model family deployed under the provider’s coding-plan configuration (snapshot September 2026). The harness does not expose temperature or top-p control, so provider defaults applied. The vision stage had no vision-capable model in this deployment and executed its documented non-visual degradation in every run (script-signed skeleton visual analysis with caption-based metadata fallback). The execution window was 17–18 September 2026, with one graded execution per scenario plus the drawn re-execution of TEP IDV11. Repair accounting: twelve of the thirteen runs closed with zero diagnosis-layer repair rounds; the exception (TEP IDV3, the hardest fault) spent one judge-driven targeted repair round, and two further scenarios required report-layer corrections after the final audit (Section 3)—every repair re-gated to PASS with no verdict text changed. Base-model capability is a confound for any cross-system comparison: the protocol-versus-baseline contrast of Section 8 holds the prompt regime constant but not the model. The pre-registered ablation matrix targeting this attribution question has been partially executed (Section 8.3): a single LLM prompted with the identical blinded statistical digest, a classical PCA baseline on all twelve scenarios, and a FaultExplainer-protocol replication on the same model family. The engine-substitution arm and the filter-disabled arm remain future work. The 18 skill packages (the prompts executed by each agent) are released with the repository; Appendix [Appendix A](#) indexes the released per-scenario artifacts.

7. Results

7.1. Anatomy of One Execution

Figure 3 makes the object of evaluation concrete before any aggregate is reported. It retraces one complete execution—TEP IDV4, the fault whose evidence chain Section 8.5 examines in detail—exactly as the released run directory records it: the fingerprint written at ingestion, the ontology’s variable semantics and its declared unmeasured channels, the statistical digest that found the recording’s one large sustained shift—a cooling-valve step, the six hypotheses with four quantitatively eliminated alternatives, the parallel reviews, the endorsed report, and the finalisation gate that turns the run into a gradeable result. Three properties of this trace carry the rest of the paper. Every decision-bearing output is a *file*, not a conversational turn: the diagnostician argues from the statistical digest, the judge and the pre-auditor recompute their spot checks from the same artifacts, and no stage can rewrite another’s output. The gates bind each transition: ontology before statistics, statistics before hypotheses, two independent reviews before the report is endorsed, and a mechanical inventory before the run counts as executed at all. And the trace is ordinary: this run cleared its diagnosis gate in the first pass and needed no repair round—true of



Figure 3: Anatomy of one complete execution (TEP IDV4, the reactor-cooling-step scenario of Section 8.5), retraced from the released run directory. Each row is one pipeline stage: what the stage did, the decision-bearing content of its artifact (abbreviated; the exact strings are in the released artifacts), and the checkpoint its transition must satisfy (Table 3). Blue chips are agent stages, grey chips deterministic scripts; Steps 5a/5b executed as two concurrent sessions. The bounded repair edge (judge < 90, dashed in Fig. 2) was not triggered in this run. The Step-9 PASS is the execution proof that contract C4 requires before a scenario may be graded (Section 5).

twelve of the thirteen released runs, the exception being the judge-driven repair on TEP IDV3 (Section 6). The remainder of this section aggregates the twelve executions this procedure produced.

7.2. Overall Performance

Table 6 reports the headline numbers. Executing the full pipeline under truth isolation (one recorded execution per scenario), IDD resolves **4/9** fault scenar-

ios to a single root cause (Top-1 = 44.4%, Wilson 95% CI [18.9%, 73.3%]); in the remaining five the data cannot separate the surviving mechanisms, and the pipeline returns capped `COMPETING_SET` verdicts—in every one of the nine scenarios the true mechanism is named within the primary finding or the surviving-hypothesis set (top- k = 9/9), so the pipeline’s failures on documented faults are failures of *resolution*, not of *localisation*. The keyword instrument needs three disclosures here, because one of the four resolved verdicts sits close to its grading tokens. The SKAB valve scenario’s keyword set names the valve in a recording whose loop definitionally contains a valve; excluding it and batch 93, whose set contains the generic token “deviation”, leaves Top-1 at 3/7 over the remainder, so the resolution pattern does not depend on the near-tautological tokens (top- k on the same clean subset is 7/7); under a branch-localised top- k reading, IDV3 counts as a miss and top- k is 8/9. Batch 93 deserves the fuller statement: its verdict is genuinely execution-side and hits the truth set on *actuator*—plus *heating* and *deviation*—but the diagnosis pins the cooling-water actuator at minimum flow while the documented deviation names the acid-dosing and heating-water flows, so under an identity-strict reading that verdict does not score and Top-1 over the nine faults is 3/9 (Wilson CI [12.1%, 64.6%]). The batch-93 discount does not propagate to top- k at the family level, which stays 9/9: set membership is exactly the mechanism-family instrument, and batch 93’s execution-side family match is what that instrument measures. The branch-localised reading of Section 6 is stricter on the TEP side and does propagate: it discounts IDV3’s stripper-branch membership, giving top- k 8/9. Throughout, a hit certifies mechanism-family agreement, never mechanism identity. Under leave-one-token-out re-scoring of the released keyword-hit records, Top-1 moves to 3/9 only if the single token “header”—IDV7’s documented mechanism word—is removed (IDV1’s two hit tokens are spelling variants of one documented label): Top-1 is an instrument-dependent estimate bounded by 3/9–4/9 across the readings this section reports, and the token sets are released so each reading is reproducible.

Determined-verdict precision is 4/4: no verdict is at once `DETERMINED` and wrong. Under the identity-strict lens the batch-93 discount propagates here too: precision becomes 3/4 and overconfidence at least one—the asymmetry is reported rather than quoted in each metric’s most favourable lens. All 3 control scenarios pass with **zero false alarms** (two of the three under briefs that disclose the segment’s normal status; Section 6). All 12 pipeline executions pass the finalisation gate with valid event logs. The deterministic rubric averages **94.6/100**, with nine scenarios at 100. The deductions are SKAB inlet-valve throttling –15 (R4 verdict-type compliance, analysed in Section 7.4), TEP IDV1 –15, and the IndPenSim control –35. The two calibration deductions arise because reported confidence 0.91 and 0.93 sit 0.01 and 0.03 above the machine gate’s flat 0.90 non-CS ceiling: the agent-side protocol documents no DET cap, so the corpus exhibits the gap between the two cap sets that Section 4 predicts. The control additionally loses 20 points on R3 grounding, its surviving “normal operation” hypothesis not being grounded in the fault-brief numbers the check targets. The in-pipeline judge gate averages 94.7 (range 90–98; all scores are post-threshold by the repair gate’s construction), and the artifact-consistency

Table 6: Overall Results on the 12-Scenario Benchmark (full-pipeline execution; one recorded execution per scenario, plus the random-draw re-execution of Section 8.4). Rubric deductions, the two cap-set exceedances, and the identity-strict reading are analysed in Section 7.4 and Section 7.2. The precision row conditions on the pipeline’s own verdict choice; its interval is conditional on that determined set. Judge scores are post-threshold (≥ 90), averaged over $n=12$ graded scenarios, and carry no benchmark weight. Wilson 95% intervals at $z=1.96$, matching the released scorer. The top- k row counts scenarios whose surviving-hypothesis set or primary finding names the true mechanism.

Metric	Value	95% CI
Resolved Top-1 (DETERMINED and correct; $n=9$)	4/9 = 44.4%	[18.9, 73.3]
<i>Instrument readings of resolved Top-1 (Section 7.2)</i>		
Identity-strict reading (batch-93 verdict discounted)	3/9 = 33.3%	[12.1, 64.6]
Leave-one-token-out re-scoring (Section 7.2)	3/9–4/9	—
True cause within top candidates (top- k ; $n=9$)	9/9 = 100%	[70.1, 100.0]
Determined-verdict precision (correct when DETERMINED)	4/4	[51.0, 100] cond. on the determined set
Ceiling compliance, capped-verdict states (confidence $\leq$ CS cap)	5/5 (gate-enforced by construction)	—
Ceiling compliance, non-CS states vs machine gate	5/7 (two exceed by 0.01/0.03; Section 4)	—
Verdict-type compliance (type in allowed set)	8/9	[56.5, 98.0]
Capped COMPETING_SET verdicts, true mechanism ranked in all five	5 of 9 faults	—
Control pass (no false alarm; $n=3$)	3/3	[43.8, 100.0]
Protocol-compliance rubric (author-designed checklist)	94.6/100 mean	nine at 100, two at 85, one at 65
Judge gate mean (in-pipeline, post-threshold ≥ 90)	94.7	range 90–98
Pipeline finalisation gate	12/12 PASS	—
Artifact-consistency gate	verified	—

gate verifies that the evaluation is internally consistent. Fig. 4 summarises the outcomes; Section 8.3 places them against model-controlled baselines.

7.3. Per-Mechanism Breakdown and Within-Family Discrimination

Table 7 lists the nine fault scenarios with the verdict each pipeline execution actually produced, the confidence, the rubric score, and the decisive statistical signature the diagnosis rested on. Three observations matter.

Within-family form discrimination. IDV4 (step), IDV11 (random variation) and IDV14 (valve stiction) all perturb the same physical channel—reactor cooling water—yet produce three distinct statistical forms that the pipeline separated, and resolved differently for recorded reasons. IDV4 (step) yields a cooling-triad step fingerprint (reactor temperature spike $z=+10.5$ with a sustained $+6.8\sigma$ valve step and a flat loop-external channel) whose sub-attribution, unmeasured inlet-temperature step versus sudden exchanger-UA loss, the data cannot separate; the verdict is a capped COMPETING_SET. IDV11 (random variation) yields a pure variance-burst signature with *frozen means* (valve-position

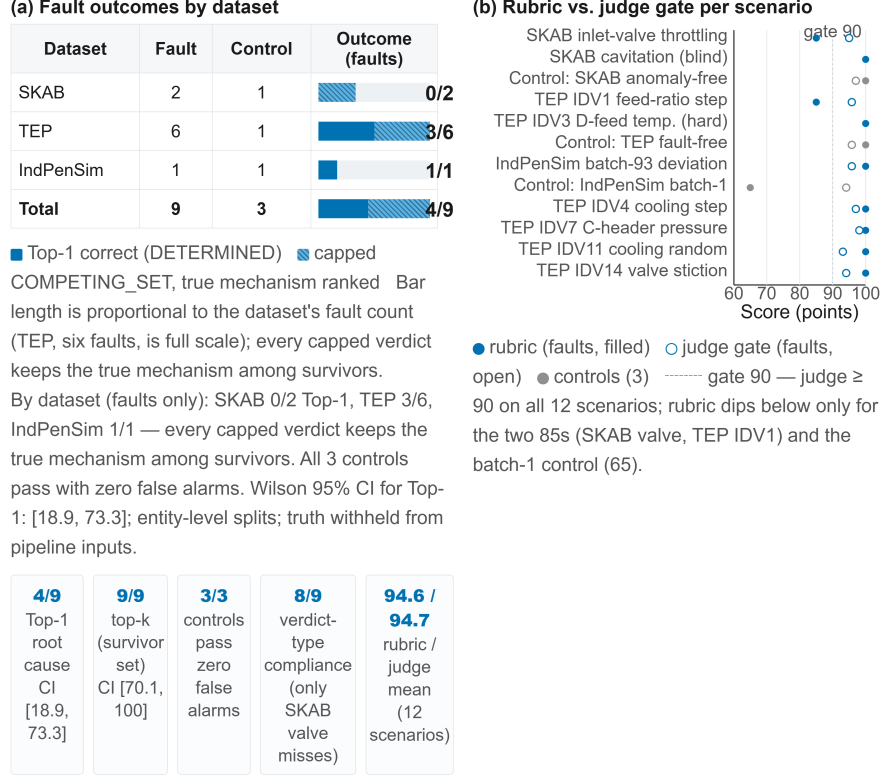


Figure 4: Benchmark outcomes under full-pipeline execution. (a) Fault outcomes by dataset: Top-1-correct DETERMINED verdicts (dark blue) and capped COMPETING_SET verdicts whose survivors rank the true mechanism (hatched sky blue)—no scenario’s survivors exclude the true mechanism. Bar length is proportional to the dataset’s fault count (TEP, with six faults, is full scale); the printed fraction is Top-1-correct over fault scenarios in that dataset, and all three controls pass with zero false alarms (excluded from the bars). (b) Per-scenario deterministic rubric (filled markers) against the in-pipeline judge gate (open markers); dark blue = fault scenarios, grey = controls; the dotted line is the judge’s 90-point gate. All 12 executions pass the finalisation gate, and the artifact-consistency gate confirms that the evaluation is internally consistent.

variance ratio 40.1, temperature 16.4, $\Delta\text{mean} \leq 0.01\sigma$, actuator leading temperature by five samples at $r = -0.64$ while the closed-loop lag-0 sign flip is barred from causal citation) and is capped again on the same inlet-versus-fouling fork. IDV14 (valve stiction) yields a self-excited stiction limit cycle (variance explosions of $\times 189/116/222$ on three synchronised channels with *frozen means*, quantised valve stepping at 633/800 samples, and a lag structure that tightens from one to five steps) and resolves to DETERMINED, because the quantised-stepping plus limit-cycle signature is an actuator fingerprint no thermal hypothesis reproduces. Recalling the textbook label “cooling-water fault” cannot produce these three different mechanism pictures; only the per-recording statistics can,

and the two capped verdicts name the one channel (cooling-water inlet temperature) whose addition would collapse their forks.

Reasoning beyond the detection limit. IDV3 and IDV4 lie beyond classical PCA monitoring and are excluded by FaultExplainer’s PCA implementation [8, 9, 1]. IDV3—commonly regarded as the hardest TEP fault [8], with a deliberately weak signature (all column mean shifts $\leq 0.92\sigma$)—terminates in a capped `COMPETING_SET` (0.55): the exclusion chain eliminates the reactor block (flat: settling $z=-0.001$), the feed/load family, and sensor artifacts, while variance inflation concentrates in the stripper steam loop (variance ratios 2.1–2.7, Levene $p<10^{-4}$, monotone stripper-temperature drift); the fork between a steam-side thermal disturbance and a stripper-feed thermal-state change cannot be resolved without unmeasured steam-header and feed-temperature measurements, and the verdict names them. The true mechanism family remains ranked among the survivors. IDV4 terminates similarly (0.63) with the heat-removal-step family top-ranked on an energy-balance necessity argument (Section 8.5). Both caps fall where classical detection and the published baseline stop; the alternative, forcing a single cause from indistinguishable evidence, is the failure mode the protocol is designed to prevent.

Blind-set robustness. The SKAB cavitation scenario is drawn from the “other” experiment group whose fault label is withheld from the scenario description. The pipeline’s surviving hypotheses are the two sides of an intermittent flow-collapse instability (76 near-zero-flow rows in forty 1–3 s bursts with the pump load essentially unchanged at 2.34 vs 2.40 A, same-window vibration rises of +5%/ +3% on two accelerometers, and stagnation heating of +0.23 K; the drive side and the voltage confound are quantitatively excluded), but the channels that would separate valve-side throttling oscillation from suction-side cavitation or air entrainment—valve position and high-frequency suction pressure—are absent, so the verdict is a capped `COMPETING_SET` (0.60) with the cavitation-type family ranked among the survivors (second of two, behind the valve-side hypothesis), naming the missing channels.

7.4. Ceiling Compliance and False-Alarm Suppression

Figure 5 plots the reported confidence per scenario. Fault-scenario confidences span 0.55–0.91; the five `COMPETING_SET` verdicts sit in the 0.55–0.63 band because the protocol caps them, and each names its discriminating measurement (valve-position/command channel and pressure direction; suction-side high-frequency pressure; cooling-water inlet temperature for both cooling-loop forks; steam-header and feed-temperature measurements for IDV3). Within the surviving sets the documented truth ranks first in two of the five capped verdicts (IDV4; IDV11 at branch level, where the cooling-water-side disturbance heads the set while its specific random-variation form ranks third of four behind the steam-branch alternative) and second in the remaining three (SKAB inlet-valve throttling and cavitation, each of two survivors; IDV3, of three, ahead of only a weak-excluded residual candidate)—the cap separates equally-evidenced mechanisms, not a truth-relevance ordering—and top- k counts set membership,

Table 7: Per-scenario results with the decisive statistical signature (abbreviated; full chains and rubric details in the released grading records). DET = DETERMINED, CS = capped COMPETING_SET; every CS verdict ranks the true mechanism among its survivors.

Scenario (mechanism)	Verdict	Rubric	Decisive signature
SKAB inlet-valve throttling	CS 0.60	85	flow plateau -3.13% ($p=5.7 \times 10^{-158}$, rows 621–927); current co-drop -0.115 A; pressure quantised
SKAB cavitation (blind set)	CS 0.60	100	intermittent collapse (76 rows, 40 bursts of 1–3 s); pump load held (2.34 vs 2.40 A); twin vibration $+5\%/+3\%$
TEP IDV1 A/C feed-ratio step	DET 0.91	85	A-feed valve/flow $+18.5\sigma$ lockstep ($r=0.9996$); C-side $-2.9/-6.9\sigma$; steam trails at lag 2
TEP IDV3 D-feed temp. (hard)	CS 0.55	100	mean shifts $\leq 0.92\sigma$; stripper steam loop var $\times 2.1\text{--}2.7$ ($p < 10^{-4}$); reactor flat; thermal fork unresolvable
TEP IDV4 cooling (step)	CS 0.63	100	cooling triad: temp $z=+10.5$, XMV_10 $+6.8\sigma$ step, loop-external flat; energy balance forces inlet/UA fork
TEP IDV7 C-header pressure	DET 0.78	100	XMV_4 $+25.1\%$ ($z=12.4$) with flow constant; gain $-20.1\% \Rightarrow \Delta P$ ratio 0.64
TEP IDV11 cooling (random)	CS 0.60	100	variance burst VR 40.1/16.4, means frozen ($\leq 0.01\sigma$); actuator leads temp 5 samples ($r=-0.64$); lag-0 flip barred
TEP IDV14 valve stiction	DET 0.80	100	VR $\times 189/116/222$, means frozen; quantised stepping 633/800; lag tightens $1 \rightarrow 5$ ($r=0.998/-0.964$)
IndPenSim batch-93	DET 0.82	100	CW loop pinned (2.0 vs 193–700 L/h) 72/72 rows $\rightarrow 301.4$ K; release $\rightarrow$ in band in 2.4 h
Controls (3)	3 DET (normal)	100/100/65	normal verdicts at 0.86–0.93; zero false alarms; raw correlations refuted as control signatures

never ordering. We report this metric as *ceiling compliance* rather than calibration; it is circular by construction, since the agent-side caps are enforced and compliance with them is a property of the protocol’s arithmetic.

The non-CS states are where the corpus contributes new evidence: the agent-side protocol documents no DETERMINED cap, while the machine grading gate sets a flat 0.90 flag bound—and two verdicts exceed it, TEP IDV1 at 0.91 and the IndPenSim control at 0.93. Both are recorded as R4 calibration deductions by the rubric and plotted in Fig. 5; neither verdict is wrong (the overconfidence metric—DETERMINED *and* wrong—never fires), but the two-point gap between the agent-side and machine-side cap sets that Section 4 predicted is now visible in the corpus rather than hypothetical—a consistency finding about our own two cap artifacts, not external validation. A psychometric calibration study (pre-gate confidence versus outcome correctness, over repeat runs) remains future work.

Under the benchmark’s strict typing, one scenario counts as a verdict-type miss: SKAB inlet-valve throttling returns COMPETING_SET where the scenario’s allowed set admits only DETERMINED (the ground truth is known to be valve restriction). The typing is one-directional: the other eight fault sce-

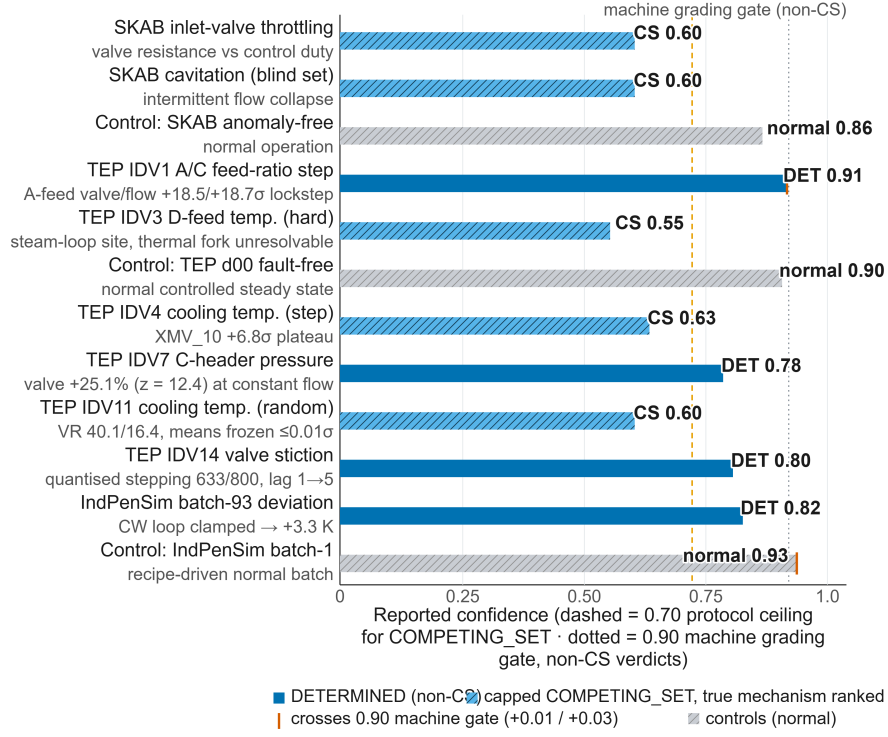


Figure 5: Reported confidence per scenario in the full-pipeline benchmark run. Solid dark blue bars are DETERMINED fault verdicts; hatched sky blue bars are the five capped COMPETING_SET verdicts (all with the true mechanism ranked among survivors); hatched grey bars are the three normal-operation controls; the dashed line is the 0.70 COMPETING_SET ceiling and the dotted line the machine grading gate’s flat 0.90 non-CS ceiling. The two bars crossing the dotted line (TEP IDV1 at 0.91, +0.01; IndPenSim control at 0.93, +0.03) mark the corpus’s one visible gap between the agent-side cap set (which documents no DET cap) and the machine gate (Section 4); the grading rubric records both as R4 calibration deductions. Each capped verdict names the discriminating measurement that would separate its surviving mechanisms; no conclusion is simultaneously DETERMINED and wrong (overconfidence 0).

narios’ allowed sets admit both states, so compliance can flag over-capping but never under-capping. The miss stands; the capped verdict is nonetheless the act the protocol demands when the valve-position and controller-command channels that would separate “valve closing” from “commanded duty reduction” are absent from the recording: the pump’s electrical load shows only a directional co-drop with the flow plateau (-0.115 A, $p=0.10$), the pressure channel is quantised at 0.327 bar (roughly an order of magnitude above the expected head change), and the flow level shift itself is small and unambiguous in timing but silent about *which* side of the restriction moved. Fig. 6 makes this scenario fully visible: the annotated recording (flow plateau in the lowest quantisation levels with the pump load essentially flat) alongside the evidence chain as the pipeline

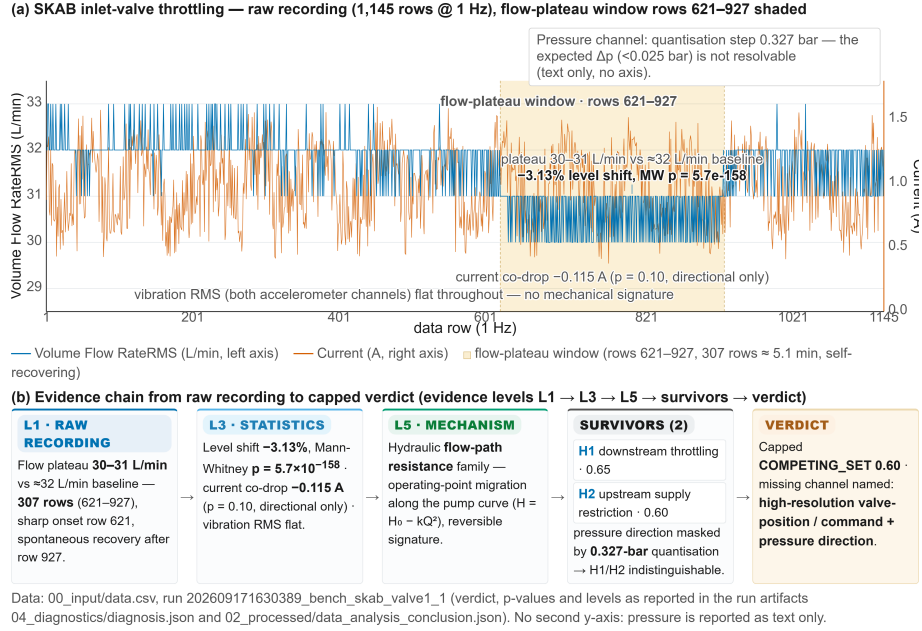


Figure 6: Worked example of a capped verdict (SKAB inlet-valve throttling, the benchmark’s single verdict-type compliance miss). Top: the recorded flow signal holds a plateau in the two lowest quantisation levels ($\approx 30\text{--}31$ vs ≈ 32 L/min; -3.13% , $p=5.7 \times 10^{-158}$; pale-amber shading marks the rows 621–927 plateau window) while the pump current co-drops only directionally (-0.115 A, $p=0.10$) and vibration stays flat; the pressure channel’s 0.327-bar quantisation (annotated in the panel) masks the head-direction measurement that would separate the two surviving mechanisms. Bottom strip: the evidence chain as the pipeline recorded it, from the raw signal through L3 statistics and the L5 mechanism to the two surviving hypotheses (downstream throttling 0.65 || upstream supply restriction 0.60) and the capped verdict with the named missing channels. Every annotation is read from the released run artifacts.

recorded it—raw signal, statistics, mechanism, the two surviving hypotheses, and the named missing channels.

Control scenarios demonstrate the complementary discipline: the normal IndPenSim batch contains recipe-dosing impulses reaching $|z|=16.2$, which the pipeline attributes to recipe steps and closed-loop control actions rather than to faults; the normal TEP run exhibits feedback signatures at $|r| \geq 0.95$ with the loop’s lag-0 peak correctly read as controller response; the normal SKAB run’s strong raw correlations ($|r|$ 0.71–0.89) are refuted as slow co-drift artifacts by the detrending and lag-compensation checks. These are the failure modes that produce false alarms in naive correlation-driven diagnosis.

8. Comparison With Published Baselines, Model-Controlled Ablation, and Verdict Stability

8.1. Register A: Per-Fault Comparison on TEP

FaultExplainer [1] reports, in its root-causes-included regime, root-cause correctness of 7/11 (63.6%, Wilson CI [35.4%, 84.8%]) for GPT-4o and 9/11 (81.8%, CI [52.3%, 94.9%]) for o1-preview on the 11 PCA-detectable TEP faults—correctness meaning the true fault or an alias in the model’s top-3, with candidates supplied in the prompt; in its no-candidate regime both models reach 8/11 under lenient “related” grading. IDD, without candidate lists and scored by strict mechanism keywords, is Top-1 correct on **3/6** TEP faults (Wilson CI [18.8%, 81.2%]) and ranks the true mechanism among the survivors in all six (top- k 6/6); on the four shared PCA-detectable faults it is correct on 3/4, where GPT-4o is 3/4 (missing IDV1, which IDD resolves) and o1-preview is 4/4 under alias-inclusive scoring. The base models differ and are not controlled, the grading protocols differ (Section 6), and the ablation of Section 8.3 shows that a bare call to the same model family already scores these documented faults at or above the pipeline (9/9 versus 4/9 resolved)—so we read the per-fault table descriptively, not as a superiority claim. Table A1 (Appendix Appendix A) transcribes the comparison; Fig. 7 breaks it down per fault against FaultExplainer’s Table 1 outcomes and PCA detectability, and Fig. 8 shows the aggregates with intervals. Most of the comparison concentrates in three outcomes:

- **IDV1 (A/C feed-ratio step):** the GPT-4o baseline mis-diagnoses this fault (o1-preview resolves it); IDD resolves it to DETERMINED (0.91) from the actuator-flow signature (A-feed valve and flow in $+18.5\sigma$ lockstep, $r=0.9996$, with the C-side feed compensating at $-2.9/ -6.9\sigma$), correctly restructuring the disturbance as an A:C supply-weight shift with the stripper steam response trailing as a downstream interlock (lag 2).
- **IDV3 and IDV4 (excluded from FE scoring):** both faults are excluded from FaultExplainer’s scoring by its PCA implementation. IDD terminates both in capped COMPETING_SET verdicts with the true mechanism ranked (Section 8.5 details the IDV4 chain).
- **IDV7, IDV11, IDV14:** IDD resolves the header-pressure form (0.78: valve-flow gain loss with flow constant $\Rightarrow$ supply-capacity drop) and the stiction form (0.80: quantised-stepping limit cycle), and caps IDV11’s random-variation form (0.60) at the inlet-vs-fouling fork—separating the three cooling-family *forms* at the level of distinct quantitative signatures (Section 7.3), a granularity below fault-identity level.

Because the scenario sets, prompt regimes, and grading protocols all differ (top-3 with alias acceptance vs. a strict single verdict; with vs. without candidate lists; different base models), we present the per-fault table as descriptive evidence and claim no significance test across systems. The comparison is also

Verification matrix — IDD (full-pipeline execution) vs FaultExplainer (arXiv:2412.14492, Table 1) with PCA detectability

TEP fault (mechanism)	FE GPT-4o	FE o1-preview	PCA detectable	Candidates in prompt	IDD verdict (conf.)	IDD decisive evidence (run artifacts)
IDV1 — A/C feed-ratio step (Stream 4)	X wrong	✓ correct	yes	yes	✓ Top-1 (0.91)	A-feed valve/flow +18.5/+18.7σ lockstep (r=0.9996); A:C supply-weight restructure
IDV3 — D feed temperature step (Stream 2)	unscored	unscored	no	yes	CS 0.55 — family ranked; site fork capped	3 mechanism families excluded; localised to the stripper steam loop (variance ×2.1–2.7); thermal fork unresolvable (no header/feed-temperature channels)
IDV4 — reactor cooling-water inlet temp. (step)	unscored	unscored	no	yes	CS 0.63 — cooling-side fork capped; top-ranked = heat-removal step	XMV_10 step +6.8σ (only sustained shift >1.2σ of 52 channels); reactor temp. spike z = +10.5, re-controlled
IDV7 — C header pressure reduced (Stream 4)	✓ correct	✓ correct	yes	yes	✓ Top-1 (0.78)	flow held at setpoint (−0.03%) while valve opens +25.1% (z = 12.4) ⇒ ΔP ratio 0.64
IDV11 — reactor cooling-water temp. (random)	✓ correct	✓ correct	yes	yes	CS 0.60 — fork capped; cooling branch top-ranked (specific form 3rd of 4)	variance burst VR 40.1/16.4 with frozen means (≤0.01σ); no step, no stiction
IDV14 — reactor cooling-water valve (stiction)	✓ correct	✓ correct	yes	yes	✓ Top-1 (0.80)	actuator limit cycle: variance ×189/116/222 with frozen means; quantised stepping 633/800; lag tightens 1→5

■ DETERMINED Top-1 · ■ capped COMPETING_SET — every amber cell keeps the true mechanism ranked among survivors (no unranked miss) · IDD summary: Top-1 3/6 (Wilson 95% CI [18.8, 81.2]); top-k 6/6 (true mechanism in all six survivor sets)

Figure 7: Per-fault comparison on the TEP subset: IDD vs. FaultExplainer (transcribed Table 1 outcomes, arXiv:2412.14492; top-3 with alias acceptance) and PCA detectability as used in FaultExplainer’s scoring (Russell et al. [8]; Chiang et al. [9]). IDD is Top-1 correct on 3/6 without candidate lists (IDV1, IDV7, IDV14) and ranks the true mechanism among the survivors in all six; the three capped COMPETING_SET verdicts name their discriminating measurements. Our own re-analysis does detect IDV4 (T^2 48.8%, SPE 100%) while IDV3 remains at the detection limit (2.9%/4.8%); because scenario sets and grading protocols differ, no significance test is claimed across systems. Colours: dark blue = IDD Top-1-correct; amber = IDD capped COMPETING_SET (true mechanism ranked); green = FE-correct; vermilion = FE-wrong; pale grey = FE-unscored and PCA-detectable “yes”; grey “no” = PCA-excluded.

asymmetric in cost: FE-style prompting runs in minutes per fault, whereas the full gated pipeline’s per-scenario agent time is on the order of two hours (Section 6).

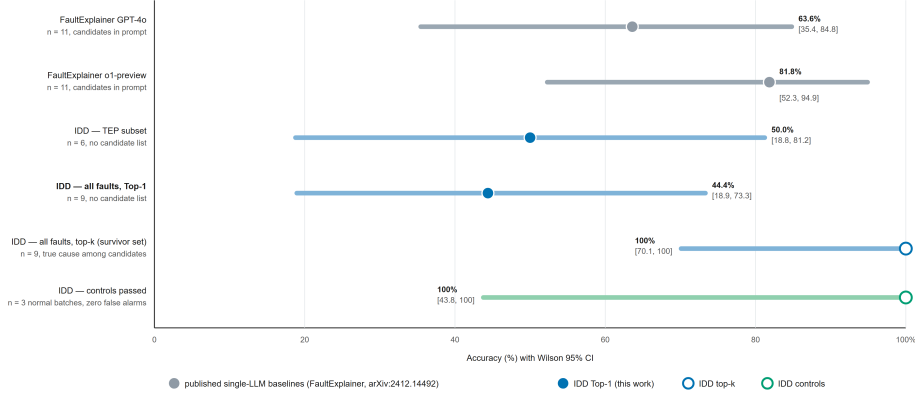


Figure 8: Register-A aggregate comparison with Wilson 95% intervals ($z=1.96$). Grey: FaultExplainer-reported single-LLM baselines ($n=11$, candidates in prompt). Blue: IDD full-pipeline execution without candidate lists—the TEP subset (3/6), all nine faults Top-1 (4/9), and top- k (9/9, every fault’s survivors rank the true mechanism). Green: the three controls (3/3, no false alarms). Interval labels for rows at 100% extend past the axis. Sets differ, so overlapping intervals are expected and no significance test is claimed; the per-fault breakdown (Fig. 7; Table A1) is the finer-grained evidence.

8.2. Register B: Reference-Only Context

For context we report sample-level results from the literature without ranking: TEP classification accuracies of 0.887–0.907 (MLP/GRU/TCN on the 28-class extended TEP taxonomy of Rieth et al. [6, 5]), anomaly-detection F1 of 0.970 (BeatGAN [13]), SKAB ensemble F1/AUC of 0.85/0.88 [51], and leakage-controlled bearing AUROC of 0.75–0.93 [18]. These systems answer “is this sample anomalous / which class”; IDD answers “what is the mechanism, how confident, with which evidence”. The dimensions are complementary: a deployment would plausibly chain a sample-level detector (first-stage alarm) with a scenario-level RCA system (this work).

8.3. Model-Controlled Baselines and Pipeline Ablation

Because the base model is a confound in any comparison with published LLM baselines (Section 6), we ran three model-controlled baselines with the *same model family and deployment* as the pipeline (Table 8; the LLM arms are recorded replays of live runs from this deployment, and the TEP fault-free control was not run bare): (i) a classical PCA baseline—normal-control training, T^2/Q detection with 99th-percentile alarms, and SPE-contribution diagnostics [9, 7]—on all twelve scenarios; (ii) a replication of FaultExplainer’s protocol on the six TEP faults, in both of FE’s prompt regimes, scored both FE-style (top-3 with alias classes) and strictly; and (iii) the pre-registered single-LLM ablation—one call to the same model family over the identical blinded statistical digest, no ontology, no pipeline—on all nine fault scenarios, scored with the same strict mechanism keywords as IDD. All three baselines are bare

Table 8: Model-Controlled Baselines on the Benchmark (same model family and deployment as the IDD pipeline; single call per scenario; answers in the released repository). The bare-LLM and FE-prompt arms are recorded replays of live runs; their feature extraction and prompt assembly, like the classical PCA arm, are deterministic re-executions. Fig. 9 shows the same arms per scenario; this table aggregates by arm.

Baseline	Candidates	Scoring	Scope	Result
Classical PCA (normal-control training)	—	detection rate	12 scen.	IDV3 T^2 2.9%, SPE 4.8%; IDV4 SPE 100% vs T^2 48.8%; controls 1%
Classical PCA (diagnosis)	—	SPE contribution	12 scen.	top-3 variables only; no mechanism verdict
FE-protocol (GLM) replication	yes	FE-style top-3 + alias	TEP 6 faults	6/6
FE-protocol (GLM) replication	yes	strict single-verdict	TEP 6 faults	6/6
FE-protocol (GLM) replication	no	strict single-verdict	TEP 6 faults	6/6
FE official prompts (GLM), T^2 -contribution features	yes	FE-style top-3 + alias	TEP 6 faults	5/6
FE official prompts (GLM), T^2 -contribution features	yes	strict single-verdict	TEP 6 faults	5/6
Single-LLM, identical blind digest (GLM)	no	strict single-verdict	9 faults	9/9
Single-LLM, controls (GLM)	no	normal verdict	2 controls	2/2, zero false alarms
IDD full pipeline	no	strict resolved Top-1	9 faults	4/9 (5 capped COMPETING_SET, true mechanism ranked in all)

single-call executions: no agent orchestration, no gates, no evidence grading—and, like the pipeline runs, each is a single recorded execution rather than a repeated sample. The classical PCA arm is computed by a released deterministic script (pure-Jacobi eigendecomposition, normal-control training, 99th-percentile alarms of the control distribution, TEP fault-window evaluation from row 161); an earlier prototype run agreed with it on every SPE statistic and control rate, with minor T^2 differences on the weakest-signature faults, and the prototype output is archived in the released repository for audit.

Three results follow, and they do not all favour the pipeline. **First**, on keyword-scored accuracy over documented public faults the bare same-model call is not merely equal to the full pipeline but ahead of it: 9/9 strict hits versus the pipeline’s 4/9 resolved (five capped verdicts)—including IDV3, plausibly by literature recall of the documented weak-signature fault, and naming valve and cavitation mechanisms on the SKAB scenarios from their signatures—and the recall caveat cuts both ways, since the pipeline’s keyword sets derive from the same documented cause table: neither arm’s absolute TEP numbers measure contamination-free ability, and the form-discrimination evidence of Section 7.3 together with the novel-disturbance suite is the clean test. The bare arm produces no false alarms on the two of the three controls the bare-LLM arm covered

(the TEP fault-free control was not run bare; the released baseline record notes the coverage gap, and only covered controls are scored). The gap between 9/9 and 4/9 is the paper’s central cost statement: the pipeline gives up resolution to gain an auditable process, and the capped verdicts are where that cost is visible. The cost is computable from the released gradings without new runs: had the protocol forced commitment to each capped verdict’s top-ranked survivor, Top-1 would be 6/9 with three wrong commitments (both SKAB capped cases and IDV3 rank the truth second)—the cap trades three false determinations for two foregone resolutions, which is the trade the three-state protocol is built to make. That the resolution loss locates in the cap-and-elimination policy rather than in orchestration is inferred from this counterfactual, not tested by an arm; the filter-disabled arm remains pre-registered.

Second, the FE official-prompt arm, running on the same GLM model, scores 5/6 on the TEP subset—above IDD’s 3/6 Top-1—with a different miss pattern: it misses IDV4 because its PCA T^2 features are dominated by feed-composition shifts that mask the cooling-water signal, while IDD caps IDV3, IDV4 and IDV11 (the two cooling-loop forks and the steam-site fork), keeping the true mechanism ranked in each. Read at IDD’s closest analogue to FE’s top-3 notion—the surviving-hypothesis set—the pipeline’s top- k on the TEP subset is 6/6.

Third, the classical PCA baseline reproduces the literature (IDV3 at the detection limit, T^2 2.9%/SPE 4.8%; IDV4 “weak T^2 , strong Q ”—SPE 100% vs T^2 48.8%) and produces no mechanism verdicts.

The three failures differ in kind, and the difference exposes what benchmark accuracy on documented faults conflates: detection ability, mechanism attribution, and epistemic discipline. The bare call produces nothing the audit plane can verify: no evidence levels, no ceiling compliance, no physics check, no execution proof, no artifact provenance—and its IDV3 answer, being correct, is exactly the kind of unverifiable guess that the three-state protocol is meant to constrain (the pipeline’s capped verdict names the steam-header and feed-temperature measurements that would settle the question).

The classical PCA baseline quantifies the detection–diagnosis gap directly: IDV3 sits at the detection limit (2.9% T^2 , 4.8% SPE), IDV4 is detected reliably only by the SPE statistic (100% vs 48.8%), and the output stops at variable contributions—no mechanism, no confidence, no verdict state. Attribution of the pipeline’s remaining margins (e.g., whether the anti-spurious filter or the ontology layer contributes) still requires the filter-disabled and engine-substitution arms, which remain future work. A reproducibility note completes the arm: after the benchmark closed, the deterministic suite arms (classical PCA, FE-protocol feature extraction, prompt assembly) were re-executed end to end—a full consecutive double run of the 45-run suite—and reproduced all 45 output files byte-for-byte with timestamps excluded (released verdict: DETERMINISTIC, 45/45 identical; the re-execution covers the deterministic suite arms, not the stochastic reasoning layer), so the baseline numbers are exactly as reproducible as the deterministic layer they run on. Fig. 9 arranges the complete comparison as a per-scenario matrix: the pipeline’s verdict column against

Per-scenario comparison — IDD (full pipeline) vs same-model bare LLM, FE-regime prompts and live PCA / FE-protocol executions

Scenario	IDD verdict (conf.)	Bare LLM (strict)	FE style (cand.)	PCA T^2 det. %	PCA SPE det. %	FE prot. det.
SKAB inlet-valve throttling	CS 0.60 · ranked	hit	-	100	100	-
SKAB cavitation (blind)	CS 0.60 · ranked	hit	-	100	100	-
Control: SKAB anomaly-free	normal 0.86	normal	-	1.0	1.0	-
TEP IDV1 feed-ratio step	Top-1 0.91	hit	hit	99.3	99.8	det.
TEP IDV3 D-feed temp. (hard)	CS 0.55 · ranked	hit	hit	2.9	4.8	no
Control: TEP fault-free (d00)	normal 0.90	-	-	1.0	1.0	no
IndPenSim batch-93 deviation	Top-1 0.82	hit	-	15.0	73.9	-
Control: IndPenSim batch-1	normal 0.93	normal	-	1.1	1.1	-
TEP IDV4 cooling step	CS 0.63 · ranked	hit	hit	48.8	100	det.
TEP IDV7 C-header pressure	Top-1 0.78	hit	hit	100	99.5	det.
TEP IDV11 cooling random	CS 0.60 · ranked	hit	hit	53.8	66.3	det.
TEP IDV14 valve stiction	Top-1 0.80	hit	hit	99.8	93.4	det.

■ IDD resolved (Top-1) ■ capped CS, mechanism ranked (5/5) ■ baseline hit / control pass ■ miss / not detected ■ PCA detection rate (%) ■ not in protocol scope

Reading. Column 1 = canonical one-execution-per-scenario grading. Bare LLM strict: 9/9 faults hit, covered controls 2/2 normal (TEP d00 control out of protocol scope). FE style with candidates (TEP 6): 6/6. FE official detection: 5/6 (IDV3 undetected; d00 "no" is the correct control outcome). PCA columns = live suite executions, sample-level T^2 /SPE detection rates (controls ≈1% = nominal false-alarm level). IDD headline: Top-1 4/9 [18.9, 73.3] · top-k 9/9 [70.1, 100] · controls 3/3, zero false alarms · TEP subset 3/6 [18.8, 81.2].

Figure 9: Per-scenario comparison matrix across the model-controlled baseline suite (all arms executed on the same prepared recordings; every cell read from the released run artifacts). Column 1: the pipeline’s verdict with reported confidence—dark blue resolved DETERMINED, hatched sky blue capped COMPETING_SET with the true mechanism ranked (all five capped verdicts carry that property), green “normal” controls. Column 2: bare single-call LLM under strict mechanism keywords (“-” on the TEP control = coverage gap, not a pass). Column 3: the same model under FaultExplainer’s candidate-list regime, FE-style scoring. Columns 4–5: classical PCA T^2 /SPE detection rates from the suite’s live re-execution on the same prepared recordings. Column 6: FaultExplainer’s six-consecutive- T^2 trigger (grey “no” on the fault-free control is the correct outcome; “_” = arm not run outside TEP)—a detection result, distinct from the scored FE-official answers of Table 8.

the two LLM-baseline arms and the three classical-detection statistics, making visible both where the systems agree (all nine faults are keyword-hit by the bare LLM, and the covered controls pass—two of three; released baseline record) and where they differ in kind (the pipeline’s five capped verdicts, all with the true mechanism ranked, and the classical detectors’ blindness to IDV3 and to IDV4’s T^2 channel).

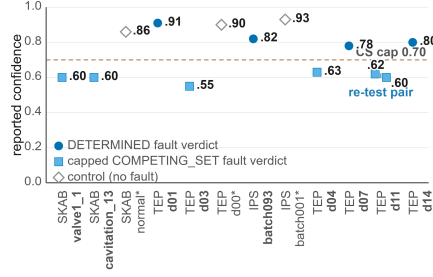
8.4. Verdict Stability: a Random-Draw Independent Re-execution

The evaluation framework formalises consistency testing as a draw-then-re-execute protocol: a scenario is selected by a *seeded uniform random draw* over the nine-fault pool (so the selection is reproducible and fixed before comparison, though the seed originates from the evaluation harness rather than an external randomness beacon—a limitation we state alongside the draw record) and then re-executed through the full gated pipeline in a fresh run directory with fresh

agent sessions (new ontology build, new statistical stage, new diagnosis, new review cycle, new HTML build and review); the draw record is append-only and replayable from the released seed. In the current corpus the draw selected TEP IDV11 (seed 1003818694; the record carries the pool, the raw uniform value, and the resulting index), and the drawn re-execution completed all nine stages end to end in ≈ 2.0 h, passing every gate (finalisation PASS). The framework’s rule is that the latest proven execution of a scenario is its canonical graded run, so the re-execution (0.60) superseded the original batch execution (0.62) in every aggregate; both are capped `COMPETING_SET` verdicts with the same signature, so the supersession changes no headline number. Fig. 10 maps the full audit.

The two executions of the drawn scenario—the corpus’s single repeat pair ($n=1$): the original batch execution 202609171639527 (run 1) and the drawn re-execution 202609172247007 (run 2, now canonical)—agree on everything the protocol measures at the mechanism level: the verdict state (capped `COMPETING_SET` in both), the implicated actuator (XMV_10, the reactor cooling-water valve), the mechanism class (environment-side cooling disturbance), and the top-ranked hypothesis (a cooling-water-side disturbance rather than a valve-internal fault), with the two sessions’ confidence estimates 0.02 apart (0.62 vs 0.60, both under the 0.65 `INDISTINGUISHABLE` cap). Both sessions also agree on the discriminating evidence: the variance-burst topology with frozen means and the actuator-leads-temperature lead structure; the retest additionally tightened the causal-direction argument (lag-5 $r=-0.636$ with the closed-loop lag-0 sign flip formally barred from causal citation) and bounded the excluded feed and condenser families more sharply. The consistency audit marks the pair `CONSISTENT`—the current corpus holds exactly one case-version pair, and within-era agreement is therefore 1/1 with zero divergences. The agreement is decided at the structured level (verdict type, equipment tag, mechanism class); the two sessions’ free-text mechanism strings share only 0.14 of cause tokens, and the re-run’s specific form ranks within the same cooling branch—wording drifts, the structure does not. One consistent pair is the beginning of a stability study, not its end. The agreement rule evaluates the coarsest structured fields (verdict type, equipment tag, mechanism class), and the drawn scenario is one whose deterministic signature—a variance burst of ratio 40 with frozen means—makes structural agreement close to mechanical, since both sessions run the same statistics stage; a draw landing on IDV3 or a SKAB case would have stressed the rule harder. With one draw, the protocol’s evidential yield is itself unmeasured. The supersession rule (latest proven execution is canonical) was fixed before comparison and will not be applied selectively to disfavour re-runs; the remaining eleven fault-or-control scenarios each hold a single current-era execution, the audit lists them as outstanding re-run contracts rather than assuming their stability, and the released consistency report enumerates them. Expanding the repeat count (all 12 scenarios, $k \geq 3$ repeats, verdict-state stability and confidence distributions) remains pre-registered in the released repository.

(a) Reported confidence, 13 current-era executions

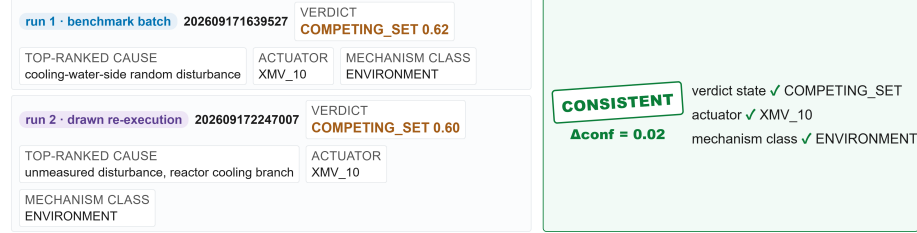


* = control (no fault injected) · dashed line = COMPETING_SET confidence cap (0.70) · all values are the run-reported confidences

consistency-audit status

scenario	audit status
SKAB valve1_1	single execution — re-run contract outstanding
SKAB cavitation_13	single execution — re-run contract outstanding
SKAB normal	single execution — re-run contract outstanding
TEP d01	single execution — re-run contract outstanding
TEP d03	single execution — re-run contract outstanding
TEP d00	single execution — re-run contract outstanding
IPS batch093	single execution — re-run contract outstanding
IPS batch001	single execution — re-run contract outstanding
TEP d04	single execution — re-run contract outstanding
TEP d07	single execution — re-run contract outstanding
TEP d11	CONSISTENT — type + actuator XMV_10 + mechanism class match, Δconf 0.02
TEP d14	single execution — re-run contract outstanding

(b) Re-tested scenario tep_d11 — run-over-run agreement (the only scenario with ≥ 2 proven executions)



Re-test draw: seeded uniform (mulberry32, seed 1003818694 — replayable); executed before any comparison — never hand-picked. Audit rule: CONSISTENT = same diagnosis type AND (same primary tag OR same mechanism class). Remaining 11 scenarios: single execution each — re-run contract outstanding.

Figure 10: Consistency audit of the current-era corpus, read from the released audit artifact. (a) Each marker is one proven execution at its reported confidence: dark blue filled circles DETERMINED fault verdicts, sky filled squares capped COMPETING_SET fault verdicts, grey open diamonds controls; TEP IDV11 carries two adjacent markers (canonical execution and drawn re-execution, $\Delta\text{confidence} = 0.02$). Dashed line: the 0.70 COMPETING_SET cap. (b) The re-test pair: verdict state, implicated actuator (XMV_10), and mechanism class match across independent sessions; audit verdict CONSISTENT. The draw was seeded and replayable (seed released), executed before any comparison. The eleven single-execution scenarios carry explicit re-run contracts rather than assumed stability.

8.5. Case Study: An Auditable Evidence Chain on a Fault Excluded from Classical Scoring

For TEP IDV4—a fault excluded from FaultExplainer’s scoring by its PCA implementation, and only weakly detected classically (T^2 48.8% vs SPE 100% in our deterministic replication) [8]—the recorded chain reads: (i) *L3 statistical evidence*: at fault onset the reactor cooling valve XMV_10 takes a sustained $+6.8\sigma$ step (41.1%→44.9% with no ramp) while reactor temperature spikes to $z=+10.5$ with a transient on its sister thermocouple ($+2.0/ - 2.3\sigma$) and ev-

ery loop-external column stays flat (the separator-outlet column at -0.08σ)—a loop-localised cooling triad; (ii) *L5 mechanism, with a necessity argument*: an energy balance on the reactor heat-removal loop shows the observed coolant-flow rise with conserved reactor temperature algebraically *requires* an upward shift in the cooling-water inlet temperature (or, indistinguishably, a sudden drop in exchanger UA)—a controller compensating exactly as the physics demands, not a sensor artifact; (iii) *eliminations*: feed-side, condenser, purge, and ramp-form alternatives are quantitatively excluded (each at $\geq 95\%$ confidence with cited contradictions), and the feed-composition channel shows no in-window signature; (iv) *conclusion*: capped `COMPETING_SET` at 0.63 (`INDISTINGUISHABLE` ceiling 0.65) between “cooling-water inlet-temperature step” (top-ranked) and “sudden UA loss”. The recording contains no cooling-water inlet measurement; the verdict names it, and the audit trail quantifies the fork: conserving the loop temperatures under the UA sub-case requires a pre-existing reactor-to-coolant temperature approach below 1.5 K (implying a coolant inlet above $\approx 93^\circ\text{C}$, against the $\approx 26\text{ K}$ approach observed in the pre-onset data), which strongly disfavours the pure-UA form—though it stays below the quantitative elimination threshold because a UA loss can redistribute the loop’s LMTD rather than move the observed temperatures, which is precisely why the verdict is a capped competing set rather than a determination. The quantification sharpens the ranking within the cap. The full chain, together with the physics-check verdict, the judge’s ten-criterion scores (97/100), and the independent HTML review (pass), is persisted in the grading record and released with the evaluation. We read the cap as the finding: on a fault the published baseline cannot even score, the pipeline localises the loop, states the physically necessary mechanism family, quantifies why the two remaining sub-mechanisms cannot be separated, and names the single measurement—the cooling-water inlet thermocouple—whose installation would collapse the set to `DETERMINED`.

9. Discussion, Limitations, and Deployment Prospects

Statistical power and single-execution evidence. With $n=9$ fault scenarios, Top-1 = 44.4% supports only a wide interval ([18.9%, 73.3%] at 95% confidence); the top- k bound (9/9, [70.1%, 100%]) is the stronger statement the current sample size can support, and it is a lower bound on resolution only because the keyword instrument is strict (Section 6). Scenario selection was purposive rather than random: the suite over-represents documented, public fault episodes, which biases difficulty downward and buys mechanism diversity across families rather than within them, so the intervals in Table 6 read as measurement uncertainty on this corpus, not sampling generalisation to a scenario population. Two qualifications attach to these figures. First, every headline number is a *single recorded execution* per scenario—except TEP IDV11, whose drawn re-execution (Section 8.4) reproduced the verdict state and mechanism at $\Delta\text{confidence } 0.02$, the only repeat measurement the corpus currently holds. Second, the bounded repair loop introduces a selection effect, since a reported verdict may be the product of up to five attempts; in this corpus, however,

twelve of thirteen runs closed without diagnosis-layer repair, the single exception being disclosed with the configuration (Section 6). The model-controlled ablation (Section 8.3) already bounds the accuracy claim (9/9 bare versus 4/9 resolved)—so the pipeline’s value on documented faults is process quality, not accuracy. Expanding the repeats (all 12 scenarios, $k \geq 3$: verdict-type stability, confidence spread, repair counts) remains pre-registered, with eleven scenarios awaiting their second in-era execution—gaps we report rather than fill; extending the corpus (the full 20-class TEP catalogue with the extended simulation dataset [52]; bearing datasets; novel-disturbance suites) is the designed path to tighter intervals.

Prompt-knowledge contamination. TEP’s documented fault mechanisms appear in textbooks and are therefore plausibly memorised by frontier models. Our protocol mitigates this at the level where memorisation actually operates—but the mitigation has a limit. The documented cause table records not only the disturbed channel but each fault’s dynamic form (IDV4 step, IDV11 random variation, IDV14 stiction), so recall of that table alone could reproduce the observed mechanism labels; the statistics-driven reading rests on the recorded reasoning chains, each of which cites the per-recording statistic that discriminates its form. Two residual channels are disclosed. First, the scenario identifier: every TEP case id and run-directory name embeds the documented mechanism and dynamic form (`tep_d14_reactor_valve_sticking` names the answer outright), so identifier recall is possible in principle across the whole documented catalogue—IDV14, one of the four resolved verdicts, is the sharpest case. The leakage sentinel machine-checks brief and user-context content, not directory names, so the identifier channel sits outside it. The joint worst case is stated rather than implied: under identifier recall on all TEP-derived outcomes together with the identity-strict batch-93 discount, resolved Top-1 could be as low as 1/9; in Table 6 and Fig. 4 the three TEP-derived resolved verdicts should be read as a recall-possible tier until the neutralised-identifier re-run executes. That re-run (six TEP scenarios under opaque hash run identifiers and identifier-stripped brief headers, ≈ 12 h of agent time by the cost accounting below) is committed as the first post-revision execution; it requires a contained harness extension (hash-id run directories) tracked in the released repository. The briefs themselves carry no fault vocabulary (leakage-sentinel checked), and the form-discrimination evidence of Section 7.3 is the main evidence-backed mitigation among the disclosed residual channels: the mechanism labels the pipeline commits to are read off per-recording statistics, which identifier recall would not require. Under the identifier channel alone, recall could supply both mechanism and dynamic form, so all four TEP-derived outcomes—including the resolved IDV1/IDV7/IDV14—carry the recall-possible tier until the re-run executes. Second, the SKAB “other”-group scenario, whose fault label is withheld entirely from all natural-language brief content, still yielded the cavitation-type mechanism ranked among the survivors of a capped verdict. The RAG channel was unavailable in every run (Section 3.2), and no L6-tagged item exists anywhere in the released corpus: no verdict rests on external evidence. A novel-disturbance suite (combination perturbations absent from the literature) is the

definitive control and remains under construction.

What the five capped verdicts mean. The five `COMPETING_SET` outcomes are the protocol working as designed, and they are also the benchmark’s most transferable finding: each verdict localises the disturbance to a subsystem, eliminates whole mechanism families with quantitative contradictions, keeps the true mechanism ranked among the survivors, and names the one measurement whose addition would collapse the set to `DETERMINED` (Section 7.4). In deployment terms these are not failures but work orders for targeted instrumentation, and the three-state protocol is what converts a diagnosis into that engineering workflow: `DETERMINED` conclusions carry mechanism chains that point to corrective action; capped conclusions carry the named channel. To keep the framing falsifiable, the released scenario definitions record the expected verdict set of every scenario, fixed before execution; SKAB inlet-valve throttling is the one scenario whose expectation was `DETERMINED`-only, and its capped verdict is counted as the benchmark’s single compliance miss rather than narrated as a success.

Inference authorship and provenance. Every scored artifact in this benchmark is authored by a gated pipeline stage—the diagnosis by the diagnostician, the quality verdict by the judge, the `ENDORSED` decision by the physical auditor, the review verdict by the independent HTML reviewer—and each carries a provenance label in the grading record. The runs reported here executed in the production dispatch form (independent per-stage sub-agent sessions; Section 5), so the reviewing stages were physically separate sessions from the authoring stages—separate in session and protocol, though sharing the one model family (Section 5). Because self-declared quality scores are not verifiable, the benchmark’s reported quality metric is the deterministic compliance rubric (Section 6)—and the rubric, the grading keywords, and the allowed verdict sets remain author-designed instruments measuring conformance to this paper’s own protocol; their release with the repository makes them auditable, but independent anchoring (the blinded expert adjudication of the released evidence chains) is the outstanding validation step—so the “process quality” margins claimed for the pipeline are architectural properties, not yet measured outcomes. A further provenance caveat: the L1–L7 levels are asserted by the reasoning stage itself; the machine gate checks ceiling arithmetic and grounding, not the correctness of a label, so a mislabelled evidence item could inflate its own ceiling—a disclosed residual risk whose mechanical binding (L1/L3 → ingestion/statistics files, L5 → ontology nodes) is a toolchain next step. An internal integrity audit that found (and removed) a scripted shortcut capable of fabricating artifacts and agent events is itself part of the released history; the completeness gate now makes that shortcut unreplayable. The audit plane’s trust boundary is internal: the event log proves process conformance, not external attestation—a compromised execution layer could still fabricate events—which is why the grader’s division of labour (scripts verify, never author) and the artifact cross-checks carry the weight. What is still missing is a comparison against *human* experts on identical digests; an expert-vs-pipeline blind test is planned.

Cost. The clean per-scenario figure for the full gated pipeline is the sequen-

tial drawn re-execution: ≈ 2.0 h of agent time end to end (117 min event span), of which the deterministic statistical stage—script-speed, seconds—is a negligible share. Per-stage wall-clock spans across the thirteen released event logs give a median full-pipeline agent span of ≈ 1.9 h (≈ 112 min: ontology ≈ 14 min, statistics ≈ 22 min, diagnosis ≈ 20 min, the two review stages ≈ 29 min combined, reporting and HTML the remaining ≈ 26 min). The twelve canonical scenarios were executed as concurrent batches under one orchestrating agent (per-run event spans of ≈ 6 h therefore reflect batching, not isolated effort; Section 6). The pipeline is orders of magnitude more expensive than a fitted classifier at inference time and far slower than single-prompt baselines. The economic comparison is only meaningful against the *engineering cost of obtaining root causes* (expert hours), and the ontology-asset layer (Section 3.2) amortises the reasoning cost across a plant’s scenario fleet.

9.1. Industrial Deployment Prospects

We separate what the benchmark and one operational deployment *demonstrate* from what the architecture *projects*.

Observed in one deployment (artifacts anonymised and not part of the released corpus): a film-line run exercised the pipeline end to end on episodic scratch defects of a biaxially-oriented polyester film line and returned NEEDS_DATA with an explicit discriminating-data list after 180 parameter-defect associations failed false-discovery-rate correction, redirecting the investigation from parameter tuning toward surface-inspection and equipment-event data; no cause was fabricated. Projected: the integration, sovereignty, and documentation claims below are design properties of the architecture, not yet benchmarked outcomes.

Integration with existing plant infrastructure. IDD consumes what plants already produce: flat exports from historians (OPC-UA/PI), LIMS quality records, and batch logs. The same pipeline runs interactively through a web application and programmatically through a versioned REST API (`/api/diagnosis` with engine selection and typed error contracts), so a deployment is an export job plus API integration rather than an instrumentation project—the integration work is real but bounded by the historian export format. Because the deterministic stage is byte-reproducible, results do not drift when the surrounding plant IT systems change.

Data-sovereign deployment. The fourteen-engine execution layer (Section 3) accepts on-premise and private-model agents through uniform dialect adapters. For operators who cannot send process data to external LLM services—defence, pharmaceuticals, semiconductors, national infrastructure—the reasoning stage can be pointed at an internally hosted model without touching the audit, evidence, or gate contracts—no non-default engine has yet executed the full gated pipeline in the released corpus, so this path is design-level, like the amortisation projection; the benchmark protocol itself executes identically against a scripted mock engine, which is how the full chain is exercised in CI.

Documentation for regulated operations. Because every conclusion is bound to graded evidence, physics checks, and a fingerprinted execution trace, the by-product of each run is an RCA report whose traceability serves the documentation expectations of regulated environments (e.g., GxP batch-deviation investigation, in the sense of ALCOA+-style data integrity [53] and GAMP 5 computerised-systems thinking [54]). This is a documentation property of the artifact, not a computerised-system validation claim; the latter would require a formal qualification exercise.

Deployment governance and knowledge compounding. IDD is positioned as decision support: a DETERMINED verdict does not authorise maintenance action—a human engineer signs off. In the current release the sign-off is procedural (no separate human-endorsement artifact exists in the run contract, and all thirteen benchmark runs were fully autonomous); we disclose this as an implementation gap, not a deployed control, and the three-state protocol is precisely the escalation interface between machine conclusion and human decision. Model-change management rides the engine layer’s version disclosure (Section 6): a provider silently updating the reasoning model changes the system, so the benchmark configuration disclosure doubles as the deployment pinning record. Finally, the ontology layer compounds knowledge across runs (Section 3.2): all thirteen benchmark runs published their ontologies to the store on completion, and every field incident becomes a candidate graded scenario, closing the loop between deployment and evaluation. In deployment the marginal cost of a diagnosis therefore falls as the fleet grows (projected: the benchmark pinned full rebuilds, so this amortisation is design-level, not measured)—an economics that sample-level detectors, which must be retrained per asset, do not share.

Scope of applicability. The method requires multivariate time-ordered process data with engineering semantics for the channels (to build the ontology) and applies to continuous, batch, and testbed processes alike—the three benchmark families were chosen to span exactly that range. It does not apply where no time-aligned process data exists, and its root-cause reach is bounded by the sensor set, as the capped verdicts demonstrate.

10. Conclusion

We presented IDD, an agentic diagnosis pipeline that turns LLM-based root-cause analysis into an auditable engineering procedure—competing hypotheses with mechanism classes, evidence levels bounding reported confidence, a four-condition anti-spurious filter, physical verification, and quality gates with bounded repair—together with a machine-checked evaluation contract for diagnosis benchmarks: truth isolation, data fingerprinting, stage-authored artifacts with provenance, and gate-graded scoring. On the 12-scenario corpus the headline outcome is deliberately modest: 4/9 faults resolved Top-1, the true mechanism ranked in every fault scenario’s surviving set (top- k 9/9; 8/9 when a survivor must carry the specific form or stream branch, not just the family), zero false alarms, five capped verdicts that each name the measurement that

would settle them, and every number backed by an execution proof. The model-controlled ablation bounds what the pipeline can claim: a bare call to the same model already exceeds its accuracy (9/9 versus 4/9), so the contribution is the auditable process rather than the accuracy. One seeded re-execution reproduced verdict state, actuator, and mechanism class within two confidence points; the deterministic suite reproduced byte-for-byte; and the per-fault TEP comparison shows IDD resolving the GPT-4o baseline’s IDV1 miss while scoring the PCA-excluded pair that baselines exclude, as capped verdicts with the mechanism family ranked. These claims have limits: resolution of documented faults is the pipeline’s weak axis by its own ablation (a bare same-model call scores 9/9 versus the pipeline’s 4/9 resolved; accuracy is not the contribution), results are single recorded executions of a stochastic reasoning layer whose configuration is disclosed and whose run identifiers carry the documented mechanism labels (so the ranking claim is not recall-blind), the machine gate’s flat 0.90 non-CS ceiling flags two verdicts (0.91 and 0.93) that the agent-side cap set permits, so the two cap sets’ gap is measured rather than hidden; and the rubric and keywords are author-designed instruments with independent expert anchoring outstanding. The evidence base is one model family at one deployment on twelve public-dataset scenarios; cross-model and cross-plant generalisation of the process guarantees (in contrast to the accuracy findings, which the ablation already bounds) remains to be demonstrated. The evaluation framework, per-scenario evidence chains, and consistency artifacts are released to support scrutiny and reuse; the pre-registered repeat and ablation studies, and the novel-disturbance suite, are the immediate next steps.

CRedit authorship contribution statement

ZhangHaoZheng: Conceptualisation, methodology, software, investigation, data curation, writing – original draft, writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Data availability

The evaluation repository accompanies this submission as supplementary material and is available for review at <https://github.com/kingdol666/>

industrial-deep-diagnostic: scenario definitions with grading keywords, expected verdict sets and per-fault literature baselines (with edit history), per-scenario blind inputs and SHA-256 fingerprints, full pipeline run directories with stage-authored artifacts and execution-event logs, grading records with provenance labels and per-scenario repair counts, the rubric and grading implementations, aggregation and consistency-verification scripts, and the Markdown/HTML scoring reports. An archived snapshot with a persistent identifier (Zenodo DOI) will be minted on acceptance. Third-party datasets are publicly available from their cited sources.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used AI-assisted coding and writing tools in order to develop the evaluation harness, execute the benchmark pipeline, and support manuscript drafting and editing. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article. The agentic system that is the *subject* of this paper (IDD) is unrelated to the writing-assistance tools used during preparation; all benchmark executions reported here were produced by the IDD pipeline itself under the released evaluation harness.

References

- [1] A. Khan, R. Nahar, H. Chen, G. E. Constante-Flores, C. Li, FaultExplainer: Leveraging large language models for interpretable fault detection and diagnosis, *Computers & Chemical Engineering* 199 (2025) 109152. [doi:10.1016/j.compchemeng.2025.109152](https://doi.org/10.1016/j.compchemeng.2025.109152).
- [2] X. Jiang, H. Xie, J. Wang, Z. Yang, Y. Zhou, L. Yao, Z. Zhu, Agentic AI for safety-aware process monitoring and fault diagnosis: A review, *Processes* 14 (13) (2026) 2112. [doi:10.3390/pr14132112](https://doi.org/10.3390/pr14132112).
- [3] M. Kano, K. Nagao, S. Hasebe, I. Hashimoto, H. Ohno, R. Strauss, B. R. Bakshi, Comparison of multivariate statistical process monitoring methods with applications to the Eastman challenge problem, *Computers & Chemical Engineering* 26 (2) (2002) 161–174. [doi:10.1016/S0098-1354\(01\)00738-4](https://doi.org/10.1016/S0098-1354(01)00738-4).
- [4] S. Tuli, G. Casale, N. R. Jennings, TranAD: Deep transformer networks for anomaly detection in multivariate time series data, *Proceedings of the VLDB Endowment* 15 (6) (2022) 1201–1214. [doi:10.14778/3514061.3514067](https://doi.org/10.14778/3514061.3514067).
- [5] V. Pozdnyakov, A. Kovalenko, I. Makarov, M. Drobyshevskiy, K. Lukyanov, Adversarial attacks and defenses in fault detection and diagnosis: A comprehensive benchmark on the tennessee eastman process, *IEEE Open*

- Journal of the Industrial Electronics Society 5 (2024) 428–440. doi:
10.1109/OJIES.2024.3401396.
- [6] C. A. Rieth, B. D. Amsel, R. Tran, M. B. Cook, Additional Tennessee Eastman process simulation data for anomaly detection evaluation (2017). doi:10.7910/DVN/6C3JR1.
 - [7] S. J. Qin, Survey on data-driven industrial process monitoring and diagnosis, Annual Reviews in Control 36 (2) (2012) 220–234. doi:10.1016/j.arcontrol.2012.09.004.
 - [8] E. L. Russell, L. H. Chiang, R. D. Braatz, Data-driven Methods for Fault Detection and Diagnosis in Chemical Processes, Springer, London, 2000. doi:10.1007/978-1-4471-0409-4.
 - [9] L. H. Chiang, E. L. Russell, R. D. Braatz, Fault Detection and Diagnosis in Industrial Systems, Springer, London, 2001. doi:10.1007/978-1-4471-0347-9.
 - [10] T. Yuan, S. J. Qin, Root cause diagnosis of plant-wide oscillations using Granger causality, Journal of Process Control 24 (2) (2014) 450–459. doi:10.1016/j.jprocont.2013.11.009.
 - [11] M. Bauer, J. W. Cox, M. H. Caveness, J. J. Downs, N. F. Thornhill, Finding the direction of disturbance propagation in a chemical process using transfer entropy, IEEE Transactions on Control Systems Technology 15 (1) (2007) 12–21. doi:10.1109/TCST.2006.883234.
 - [12] J. J. Downs, E. F. Vogel, A plant-wide industrial process control problem, Computers & Chemical Engineering 17 (3) (1993) 245–255. doi:10.1016/0098-1354(93)80018-I.
 - [13] F. Hartung, B. J. Franks, T. Michels, D. Wagner, P. Liznerski, S. Reithermann, S. Fellenz, F. Jirasek, M. Rudolph, D. Neider, H. Leitte, C. Song, B. Kloepper, S. Mandt, M. Bortz, J. Burger, H. Hasse, M. Kloft, Deep anomaly detection on Tennessee Eastman process data, Chemie Ingenieur Technik 95 (7) (2023) 1077–1082. doi:10.1002/cite.202200238.
 - [14] V. Venkatasubramanian, R. Rengaswamy, K. Yin, S. N. Kavuri, A review of process fault detection and diagnosis Part I: Quantitative model-based methods, Computers & Chemical Engineering 27 (3) (2003) 293–311. doi:10.1016/S0098-1354(02)00160-6.
 - [15] V. Venkatasubramanian, R. Rengaswamy, S. N. Kavuri, A review of process fault detection and diagnosis: Part II: Qualitative models and search strategies, Computers & Chemical Engineering 27 (3) (2003) 313–326. doi:10.1016/S0098-1354(02)00218-X.

- [16] V. Venkatasubramanian, R. Rengaswamy, S. N. Kavuri, K. Yin, A review of process fault detection and diagnosis: Part III: Process history based methods, *Computers & Chemical Engineering* 27 (3) (2003) 327–346. doi:[10.1016/S0098-1354\(02\)00219-1](https://doi.org/10.1016/S0098-1354(02)00219-1).
- [17] R. Isermann, Model-based fault-detection and diagnosis – status and applications, *Annual Reviews in Control* 29 (1) (2005) 71–85. doi:[10.1016/j.arcontrol.2004.12.002](https://doi.org/10.1016/j.arcontrol.2004.12.002).
- [18] J. P. Vieira, V. A. Bauler, R. K. Rosa, D. Silva, Towards a more realistic evaluation of machine learning models for bearing fault diagnosis, *Mechanical Systems and Signal Processing* 258 (2026) 114640. doi:[10.1016/j.ymssp.2026.114640](https://doi.org/10.1016/j.ymssp.2026.114640).
- [19] J. Hendriks, P. Dumond, D. A. Knox, Towards better benchmarking using the CWRU bearing fault dataset, *Mechanical Systems and Signal Processing* 169 (2022) 108732. doi:[10.1016/j.ymssp.2021.108732](https://doi.org/10.1016/j.ymssp.2021.108732).
- [20] M. Iri, K. Aoki, E. O’Shima, H. Matsuyama, An algorithm for diagnosis of system failures in the chemical process, *Computers & Chemical Engineering* 3 (1–4) (1979) 489–493. doi:[10.1016/0098-1354\(79\)80079-4](https://doi.org/10.1016/0098-1354(79)80079-4).
- [21] F. Tao, H. Zhang, A. Liu, A. Y. C. Nee, Digital twin in industry: state-of-the-art, *IEEE Transactions on Industrial Informatics* 15 (4) (2019) 2405–2415. doi:[10.1109/TII.2018.2873186](https://doi.org/10.1109/TII.2018.2873186).
- [22] W. Gong, S. Qiao, C. Cao, S. Tan, J. Ye, H. Liu, S. Chen, X. Wang, Harnessing collective intelligence of multi-agent LLM systems for sensor failure reasoning in smart manufacturing, *Journal of Industrial Information Integration* 49 (2026) 101012. doi:[10.1016/j.jii.2025.101012](https://doi.org/10.1016/j.jii.2025.101012).
- [23] C. Constantinides, D. Patel, S. Lin, C. Guerrero, S. D. Patil, et al., FailureSensorIQ: A multi-choice QA dataset for understanding sensor relationships and failure modes (2025). [arXiv:2506.03278](https://arxiv.org/abs/2506.03278), doi:[10.48550/arXiv.2506.03278](https://doi.org/10.48550/arXiv.2506.03278).
- [24] C. Liu, J. Song, D. Tang, L. Wang, H. Zhu, Q. Cai, Probing a novel machine tool fault reasoning and maintenance service recommendation approach through data-knowledge empowered LLMs integrated with AR-assisted maintenance guidance, *Advanced Engineering Informatics* 66 (2025) 103460. doi:[10.1016/j.aei.2025.103460](https://doi.org/10.1016/j.aei.2025.103460).
- [25] Y. Liu, Z. Xu, J. Zhao, C. Song, Z. He, Spatial-temporal adaptive causality graph-based fault root cause location method for time-varying industrial process, *Advanced Engineering Informatics* 68 (2025) 103765. doi:[10.1016/j.aei.2025.103765](https://doi.org/10.1016/j.aei.2025.103765).
- [26] W. Yue, J. Chai, X. Wan, Y. Xie, X. Chen, W. Gui, Root cause analysis for process industry using causal knowledge map under large group

- environment, *Advanced Engineering Informatics* 57 (2023) 102057. doi:[10.1016/j.aei.2023.102057](https://doi.org/10.1016/j.aei.2023.102057).
- [27] Z. Liu, H. Zheng, H. Liu, W. Jia, J. Tan, Label-free evaluation for performance of fault diagnosis model on unknown distribution dataset, *Advanced Engineering Informatics* 62 (2024) 102912. doi:[10.1016/j.aei.2024.102912](https://doi.org/10.1016/j.aei.2024.102912).
 - [28] K. M. Alsaif, A. A. Albeshri, M. A. Khemakhem, F. E. Eassa, Multi-modal large language model-based fault detection and diagnosis in context of industry 4.0, *Electronics* 13 (24) (2024) 4912. doi:[10.3390/electronics13244912](https://doi.org/10.3390/electronics13244912).
 - [29] S. Merkelbach, W. Hasenauer, Using multi-modal LLMs to create models for fault diagnosis, in: *DX 2024 – 35th International Workshop on Principles of Diagnosis (OASICS)*, 2024, pp. 31:1–31:8. doi:[10.4230/OASICS.DX.2024.31](https://doi.org/10.4230/OASICS.DX.2024.31).
 - [30] B. Zhou, X. Li, T. Liu, K. Xu, W. Liu, J. Bao, CausalKGPT: Industrial structure causal knowledge-enhanced large language model for cause analysis of quality problems in aerospace product manufacturing, *Advanced Engineering Informatics* 59 (2024) 102333. doi:[10.1016/j.aei.2023.102333](https://doi.org/10.1016/j.aei.2023.102333).
 - [31] L. Lin, S. Zhang, S. Fu, Y. Liu, FD-LLM: Large language model for fault diagnosis of complex equipment, *Advanced Engineering Informatics* 65 (2025) 103208. doi:[10.1016/j.aei.2025.103208](https://doi.org/10.1016/j.aei.2025.103208).
 - [32] S. Wen, F. Li, W. Zhuang, X. Pan, W. Yu, J. Bao, X. Li, Leveraging large language models for Human-Machine collaborative troubleshooting of complex industrial equipment faults, *Advanced Engineering Informatics* 65 (2025) 103235. doi:[10.1016/j.aei.2025.103235](https://doi.org/10.1016/j.aei.2025.103235).
 - [33] Q. Xu, G. Wen, Z. Lei, S. Gu, Y. Su, Z. Zhang, X. Chen, Deep digital twin-powered large vision-language model for multi-scenario industrial fault diagnosis, *Advanced Engineering Informatics* 69 (2026) 103997. doi:[10.1016/j.aei.2025.103997](https://doi.org/10.1016/j.aei.2025.103997).
 - [34] A. Wei, O. Fink, AgentRCA: Agentic root cause analysis through evidence-grounded reasoning (2026). [arXiv:2607.22385](https://arxiv.org/abs/2607.22385), doi:[10.48550/arXiv.2607.22385](https://doi.org/10.48550/arXiv.2607.22385).
 - [35] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, AutoGen: Enabling next-gen LLM applications via multi-agent conversation (2023). [arXiv:2308.08155](https://arxiv.org/abs/2308.08155), doi:[10.48550/arXiv.2308.08155](https://doi.org/10.48550/arXiv.2308.08155).
 - [36] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, ReAct: Synergizing reasoning and acting in language models, in: *International*

- Conference on Learning Representations (ICLR), 2023. [doi:10.48550/arXiv.2210.03629](https://doi.org/10.48550/arXiv.2210.03629).
- [37] R. Parasuraman, T. B. Sheridan, C. D. Wickens, A model for types and levels of human interaction with automation, *IEEE Transactions on Systems, Man, and Cybernetics—Part A: Systems and Humans* 30 (3) (2000) 286–297. [doi:10.1109/3468.844354](https://doi.org/10.1109/3468.844354).
 - [38] J. D. Lee, K. A. See, Trust in automation: designing for appropriate reliance, *Human Factors* 46 (1) (2004) 50–80. [doi:10.1518/hfes.46.1.50_30392](https://doi.org/10.1518/hfes.46.1.50_30392).
 - [39] R. J. Heuer, *Psychology of Intelligence Analysis*, Center for the Study of Intelligence, Central Intelligence Agency, Washington, DC, 1999.
 - [40] A. B. Hill, The environment and disease: association or causation?, *Proceedings of the Royal Society of Medicine* 58 (1965) 295–300. [doi:10.1177/003591576505800503](https://doi.org/10.1177/003591576505800503).
 - [41] C. K. Chow, On optimum recognition error and reject tradeoff, *IEEE Transactions on Information Theory* 16 (1) (1970) 41–46. [doi:10.1109/TIT.1970.1054406](https://doi.org/10.1109/TIT.1970.1054406).
 - [42] A. N. Angelopoulos, S. Bates, Conformal prediction: A gentle introduction, *Foundations and Trends in Machine Learning* 16 (4) (2023) 494–591. [doi:10.1561/22000000101](https://doi.org/10.1561/22000000101).
 - [43] F. Doshi-Velez, B. Kim, Towards A Rigorous Science of Interpretable Machine Learning, *arXiv preprint arXiv:1702.08608* (2017).
 - [44] J. K. Scott, R. Findeisen, R. D. Braatz, D. M. Raimondo, Input design for guaranteed fault diagnosis using zonotopes, *Automatica* 50 (6) (2014) 1580–1589. [doi:10.1016/j.automatica.2014.03.016](https://doi.org/10.1016/j.automatica.2014.03.016).
 - [45] J. Morbach, A. Wiesner, W. Marquardt, OntoCAPE—a (re)usable ontology for computer-aided process engineering, *Computers & Chemical Engineering* 33 (10) (2009) 1546–1556. [doi:10.1016/j.compchemeng.2009.01.019](https://doi.org/10.1016/j.compchemeng.2009.01.019).
 - [46] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: *Advances in Neural Information Processing Systems* 33 (NeurIPS 2020), 2020, arXiv:2005.11401.
 - [47] I. D. Katser, V. O. Kozitsin, Skoltech anomaly benchmark (SKAB), <https://www.kaggle.com/dsv/1693952> (2020). [doi:10.34740/KAGGLE/DSV/1693952](https://doi.org/10.34740/KAGGLE/DSV/1693952).

- [48] P. R. Lyman, C. Georgakis, Plant-wide control of the Tennessee Eastman problem, *Computers & Chemical Engineering* 19 (3) (1995) 321–331. doi:[10.1016/0098-1354\(94\)00057-U](https://doi.org/10.1016/0098-1354(94)00057-U).
- [49] S. Goldrick, A. Ştefan, D. Lovett, G. Montague, B. Lennox, The development of an industrial-scale fed-batch fermentation simulation, *Journal of Biotechnology* 193 (2015) 70–82. doi:[10.1016/j.jbiotec.2014.10.029](https://doi.org/10.1016/j.jbiotec.2014.10.029).
- [50] S. Goldrick, C. A. Duran-Villalobos, K. Jankauskas, D. Lovett, S. S. Farid, B. Lennox, Modern day monitoring and control challenges outlined on an industrial-scale benchmark fermentation process, *Computers & Chemical Engineering* 130 (2019) 106471. doi:[10.1016/j.compchemeng.2019.05.037](https://doi.org/10.1016/j.compchemeng.2019.05.037).
- [51] A. Iliopoulos, J. Violos, C. Diou, I. Varlamis, Detection of anomalies in multivariate time series using ensemble techniques, in: *2023 IEEE Ninth International Conference on Big Data Computing Service and Applications (BigDataService)*, 2023, pp. 1–8. doi:[10.1109/BigDataService58306.2023.00007](https://doi.org/10.1109/BigDataService58306.2023.00007).
- [52] C. Reinartz, M. Kulahci, O. Ravn, An extended tennessee eastman simulation dataset for fault detection and decision support systems, *Computers & Chemical Engineering* 149 (2021) 107281. doi:[10.1016/j.compchemeng.2021.107281](https://doi.org/10.1016/j.compchemeng.2021.107281).
- [53] Medicines and Healthcare products Regulatory Agency (MHRA), ‘gxp’ data integrity guidance and definitions, revision 1, Tech. rep., MHRA, London (2018).
- [54] ISPE, GAMP 5: A Risk-Based Approach to Compliant GxP Computerized Systems, 2nd Edition, International Society for Pharmaceutical Engineering, Tampa, FL, 2022.

Appendix A. Supplementary Tables and Released-Artifact Index

Table [A1](#) transcribes the per-fault TEP comparison discussed in Section 8. The artifacts released for every benchmark scenario are exactly the fourteen-item completeness set of Table 4, laid out one run directory per scenario in the released repository, each machine-readable and carrying the execution proofs described in Section 5. The machine-generated English benchmark report is `results/benchmark/benchmark_report_en.md`; its own appendix lists the grader-side ground truth and must not be exposed to a diagnosing agent.

Table A1: Per-Fault Outcomes on the TEP Subset (machine-readable companion to Fig. 7). FE = FaultExplainer (root-causes-included prompt, top-3 candidate list, alias hits accepted); “unscored” = excluded from FE scoring by its PCA implementation [9]. IDD: strict mechanism-keyword scoring, no candidate list; DET = DETERMINED; CS = capped COMPETING_SET (every CS verdict ranks the true mechanism among its survivors; TEP-subset top- $k = 6/6$). In FE’s additional no-candidate regime (General-Reasoning prompt, top-3, lenient “related” grading), both models score 8/11. IDD’s Top-1 summary: 3/6 (Wilson CI [18.8%, 81.2%], $n=6$); shared detectable subset 3/4.

TEP fault	FE GPT-4o	FE ol-preview	PCA detectable	IDD verdict (conf.)
IDV1 A/C feed-ratio step (Stream 4)	wrong	correct	yes	Top-1 correct DET 0.91
IDV3 D-feed temperature step	unscored	unscored	no	CS 0.55 (steam-loop site; fork named — survivor names “D/E feed temperature or stream-10 enthalpy”, the truth family inside a stripper-feed-branch hypothesis)
IDV4 reactor cooling-water temp. (step)	unscored	unscored	no	CS 0.63 (heat-removal step top-ranked; inlet/UA fork named)
IDV7 C-header pressure (Stream 4)	correct	correct	yes	Top-1 correct DET 0.78
IDV11 cooling-water temp. (random)	correct	correct	yes	CS 0.60 (random-variation form; inlet/UA fork named)
IDV14 cooling-water valve (stiction)	correct	correct	yes	Top-1 correct DET 0.80
Scoreable-set summary	7/11	9/11	—	3/6 (CI [18.8%, 81.2%], $n=6$); shared detectable subset 3/4; top- k 6/6
FE no-candidate regime (top-3, lenient)	8/11 both models	—	—	—